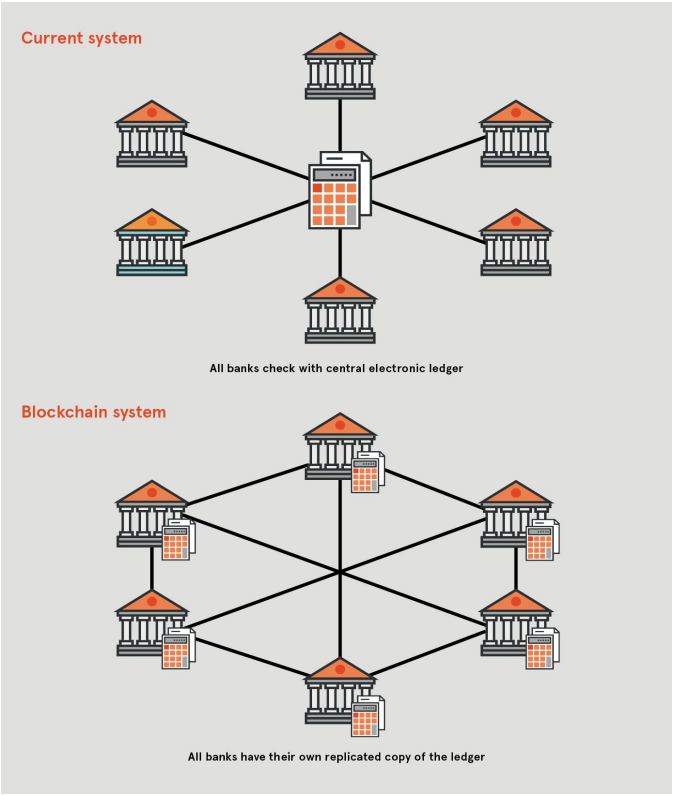
Distributed System and Consensus

WANG Ye



bitcoin value



Bitcoin System Architecture

How blockchain system appear ?



How blockchain system appear ?

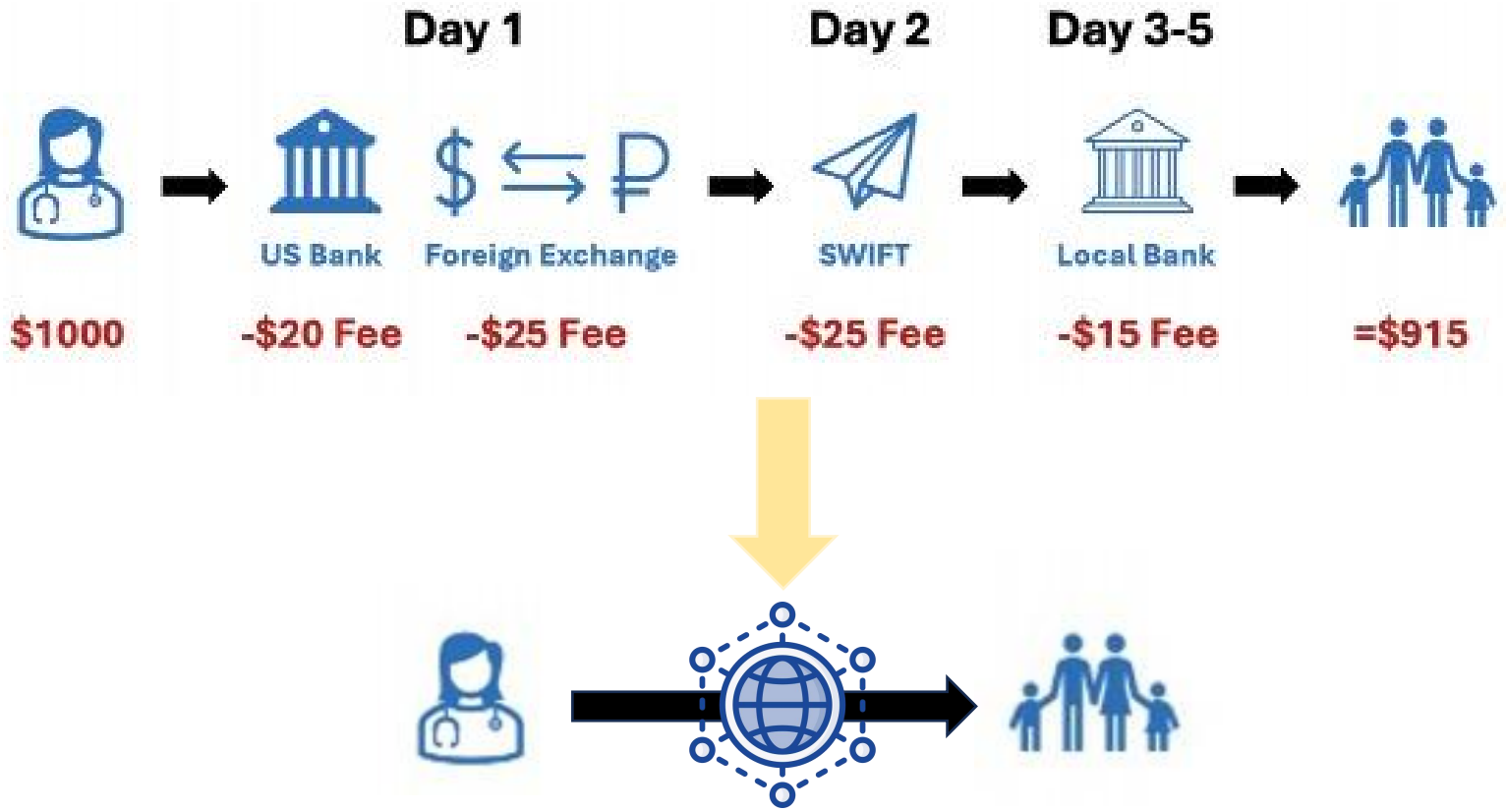


People put money in “demand deposits” for payments purposes.

Banks make loans that are partially backed by deposits.

Most of the money in circulation is privately created by commercial banks.

How blockchain system appear ?



Technology Replaces Intermediaries

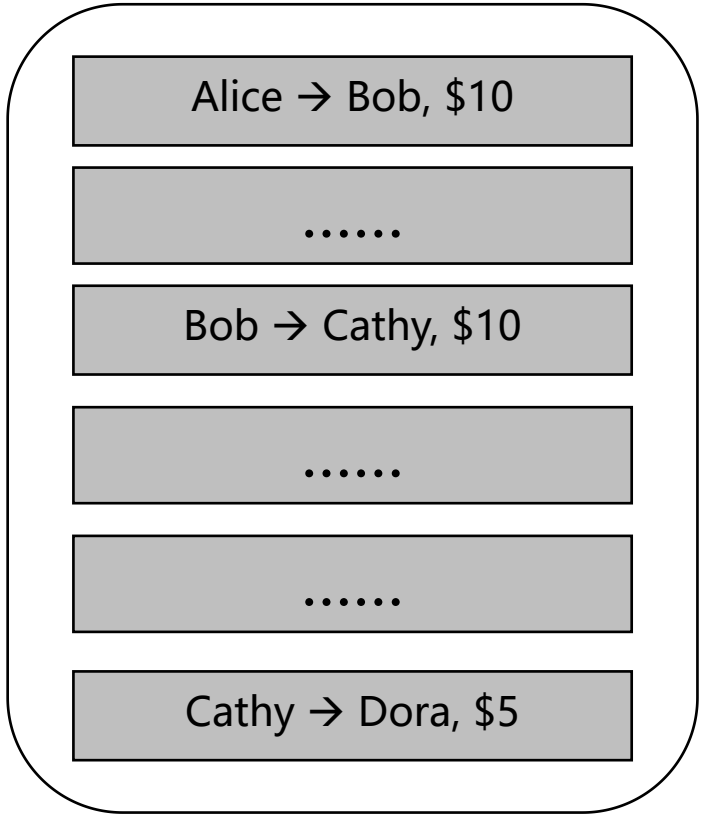
Reduce transaction costs
Faster payment rate

How blockchain system appear ?

Global Ledger

A ledger refers to a book that contains different accounts where records of **transactions** pertaining to a specific account is stored. It is also known as the book where **all transactions are stored**.

Global ledger means that all members using this ledger read and write transactions in the same ledger, preventing any centralized agent.



Ledger System

How blockchain system appear ?

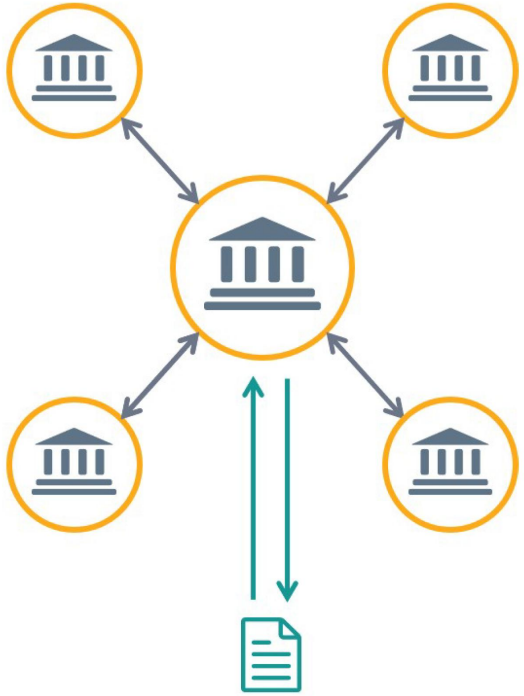
Client/Server Architecture

Data Source: powerful, reliable server

Clients request data from server

Popular Model

- WWW (HTTP)
- FTP
- Web services
- Etc.



How blockchain system appear ?

Client/Server Architecture

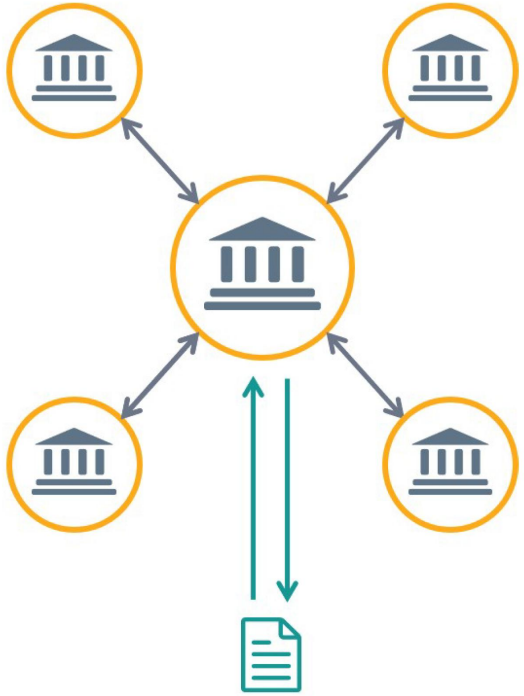
Scalability is hard to achieve

Presents a Single Point of Failure

Requires administration

Unused resources at the network edge

- CPU cycles, storage, etc.



As more clients are added, the demand on the server increases!!!

How blockchain system appear ?

Satoshi Nakamoto

Paper: **"Bitcoin: A Peer-to-Peer Electronic Cash System”** (2008)

" (Bitcoin system) is an electronic payment system based on cryptographic proof instead of trust, allowing any two willing parties to transact directly with each other without the need for a trusted third party.”

How blockchain system appear ?

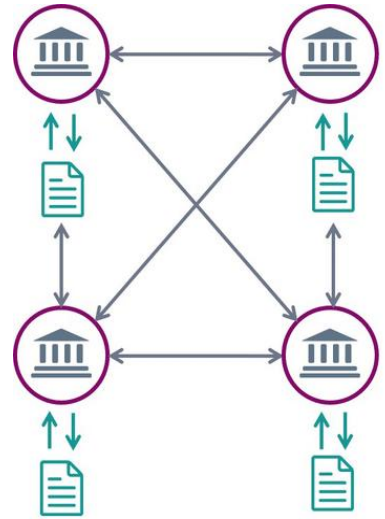
Distributed System

A distributed system architecture

- no central server
- formed by heterogeneous and unreliable nodes

Nodes are symmetric in function

- serving as clients, routers and servers
- provide and consume data
- any node can initiate a connection



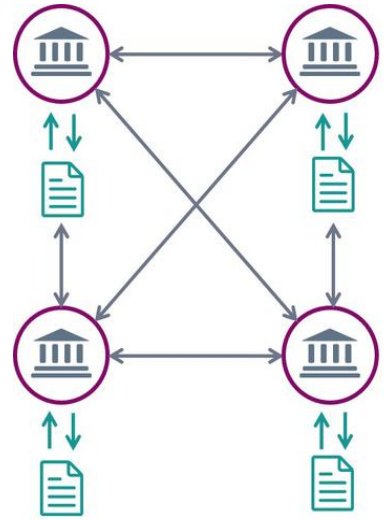
Distributed ledger

How blockchain system appear ?

Distributed Ledger

It has copies of the same global ledger replicated among several participant nodes, each with a right to control and update the ledger based on a predefined system for achieving **consensus**.

Bitcoin system uses distributed ledger to confirm that every node can have a global ledger.



Distributed ledger

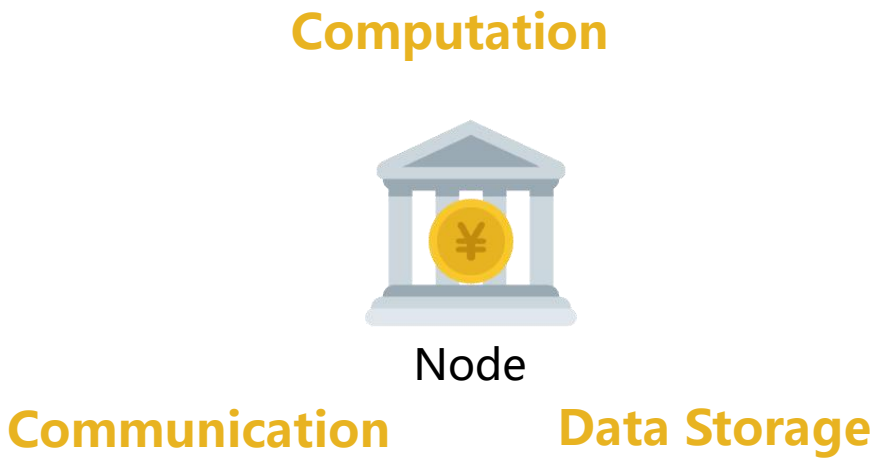
How blockchain system appear ?

Model		Application
Distributed System	⟺	Bitcoin System
Consensus	⟺	Global Ledger

Basic Concepts of Distributed Systems

(Definition 2.1) Node

- We call a single actor in the system node.



Basic Concepts of Distributed Systems

(Definition 2.1) Node

- We call a single actor in the system node.

.

- If not stated otherwise, the total number of nodes in the system is **n**.



Node

n = 3



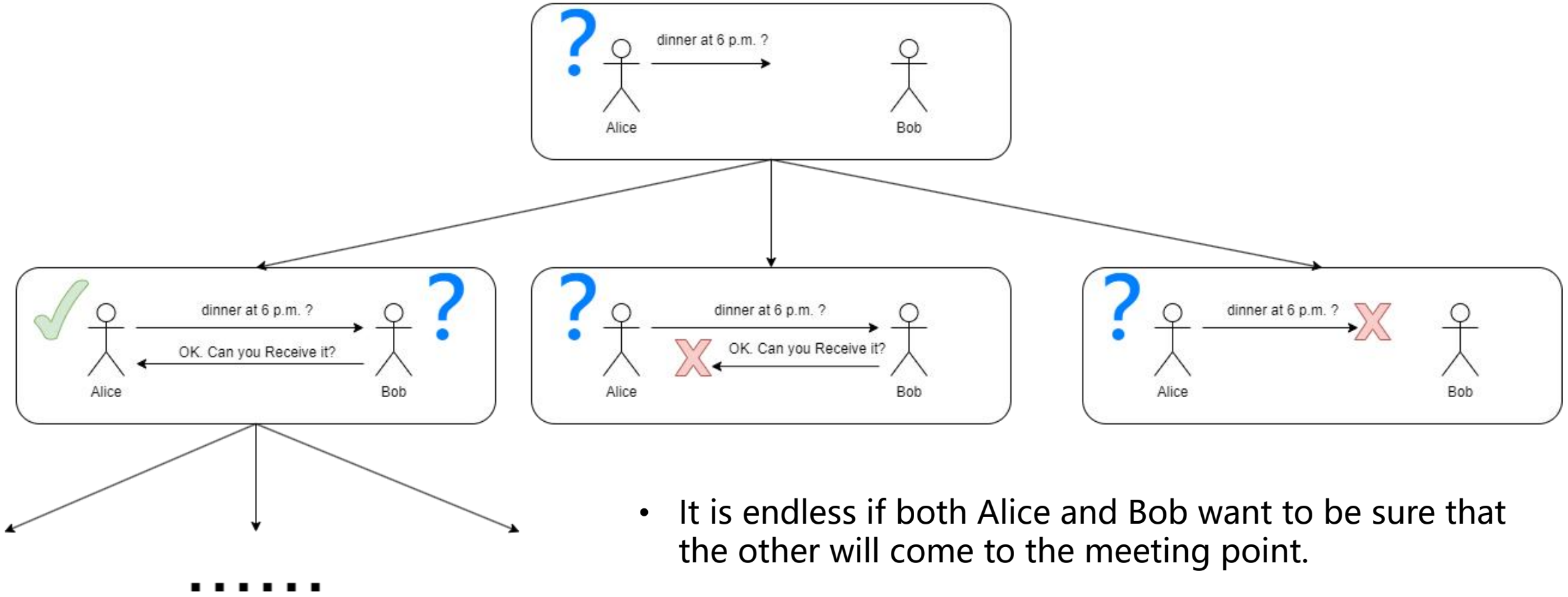
Node



Node

Consensus

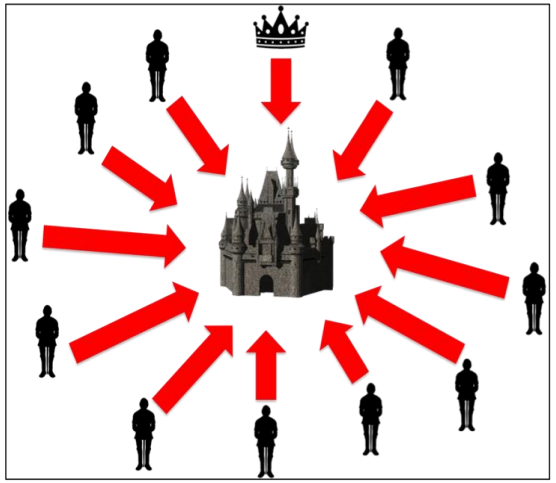
Example: Two Friends



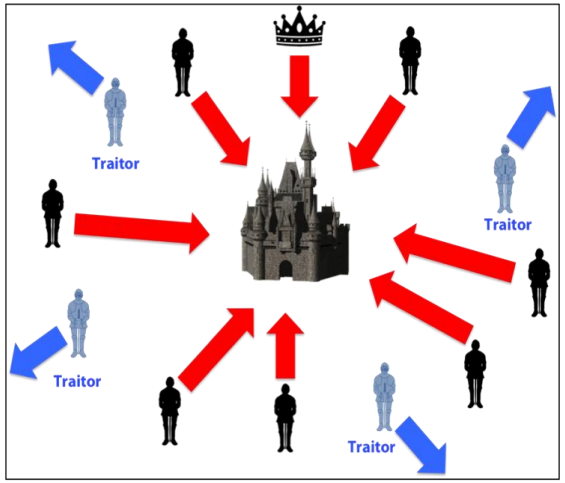
- It is endless if both Alice and Bob want to be sure that the other will come to the meeting point.

The Byzantine Generals' Problem (BGP)





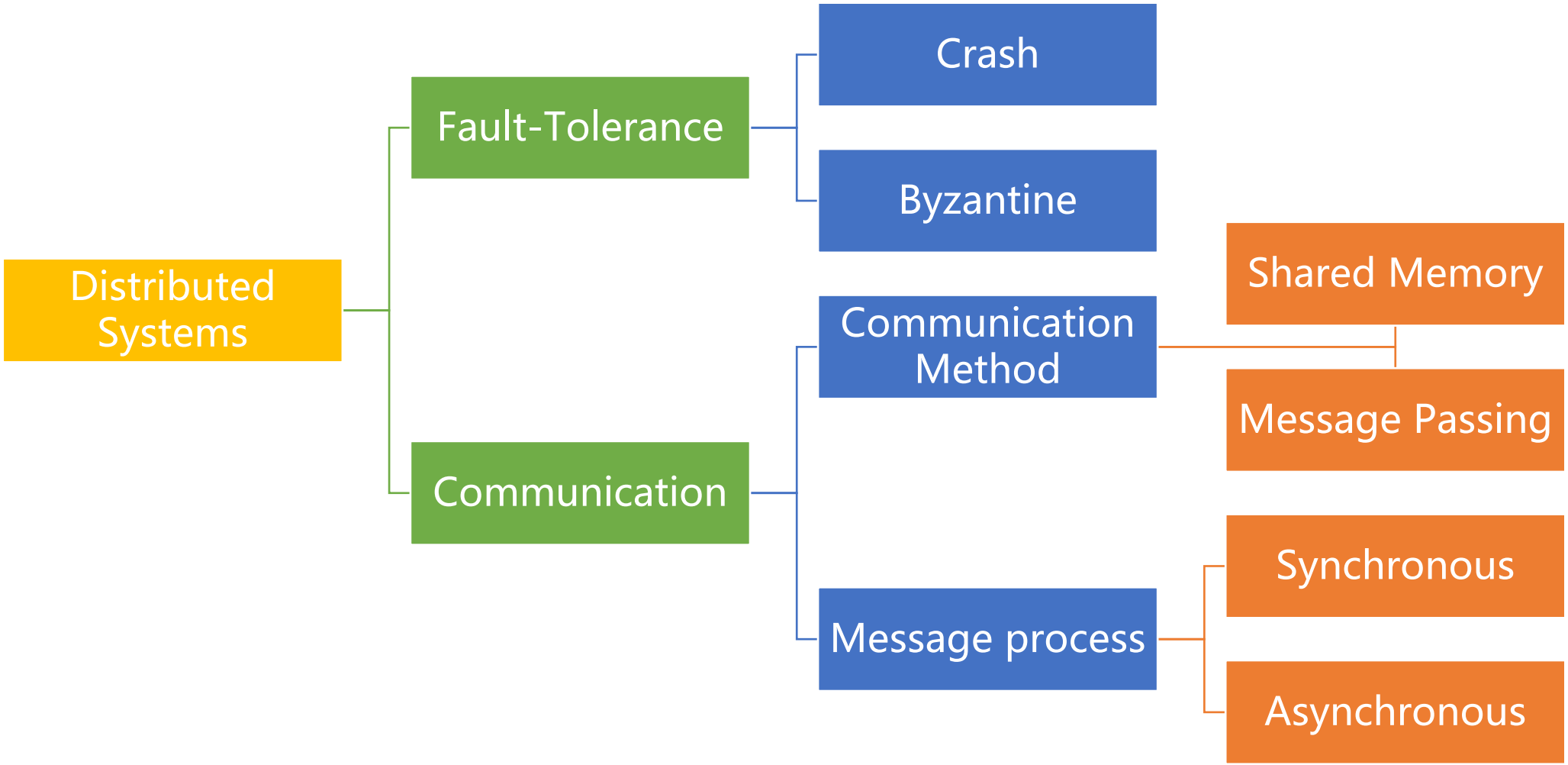
Coordinated Attack Leading to Victory



Uncoordinated Attack Leading to Defeat

The generals sent *messengers* to communicate with each other.
There could have been traitor/spies among other generals and the messengers.

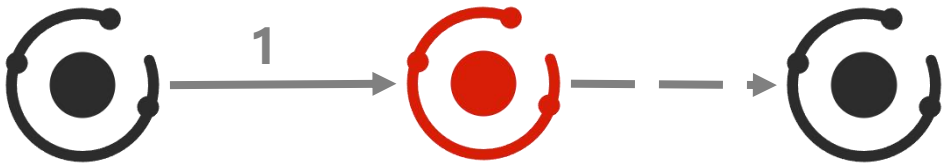
Basic Concepts of Distributed Systems



Basic Concepts of Distributed Systems

Fault-Tolerance

Fault tolerance is the process of designing a system to be able to continue operating despite errors or failures.



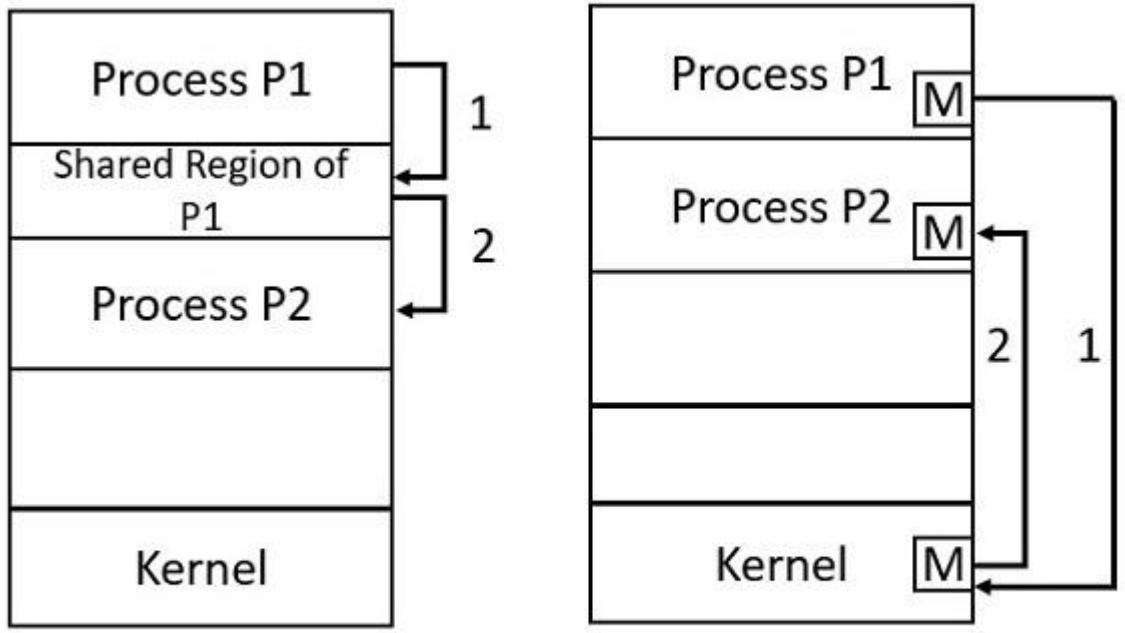
Crash Node



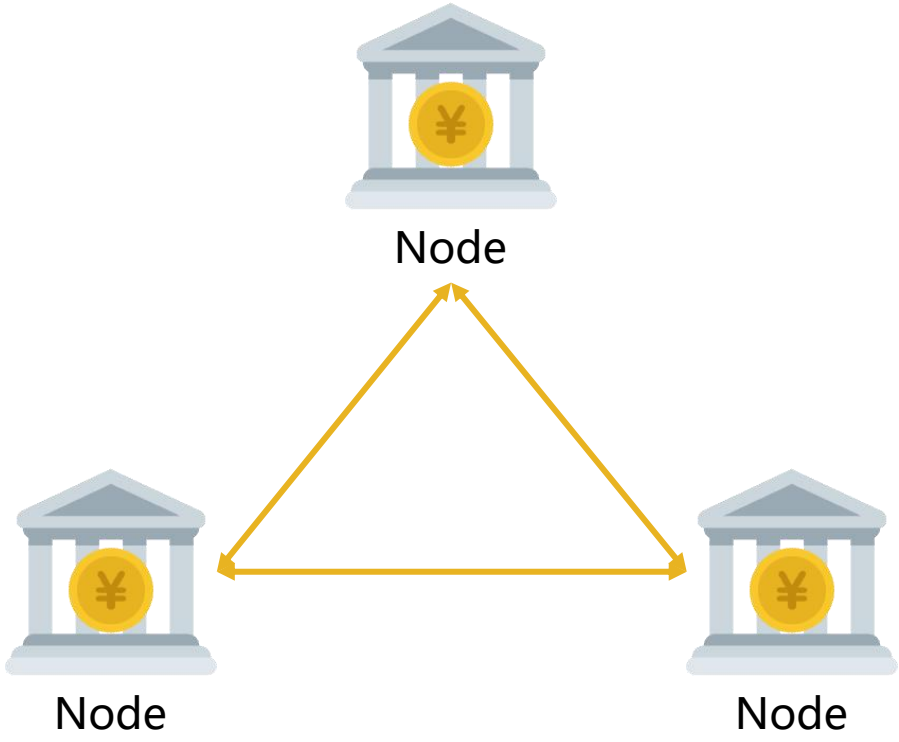
Byzantine Node

Basic Concepts of Distributed Systems

Communication Method

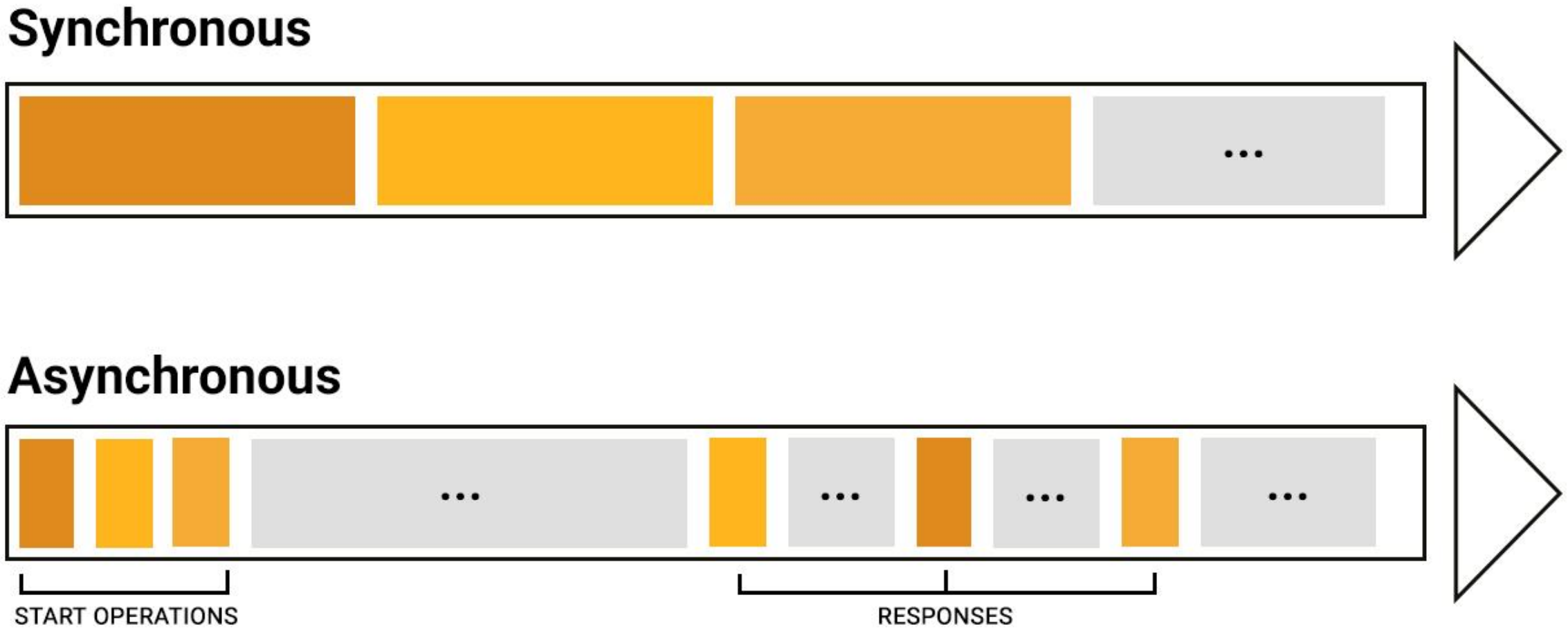


Shared Memory System Message Passing System



Basic Concepts of Distributed Systems

Messages Processing



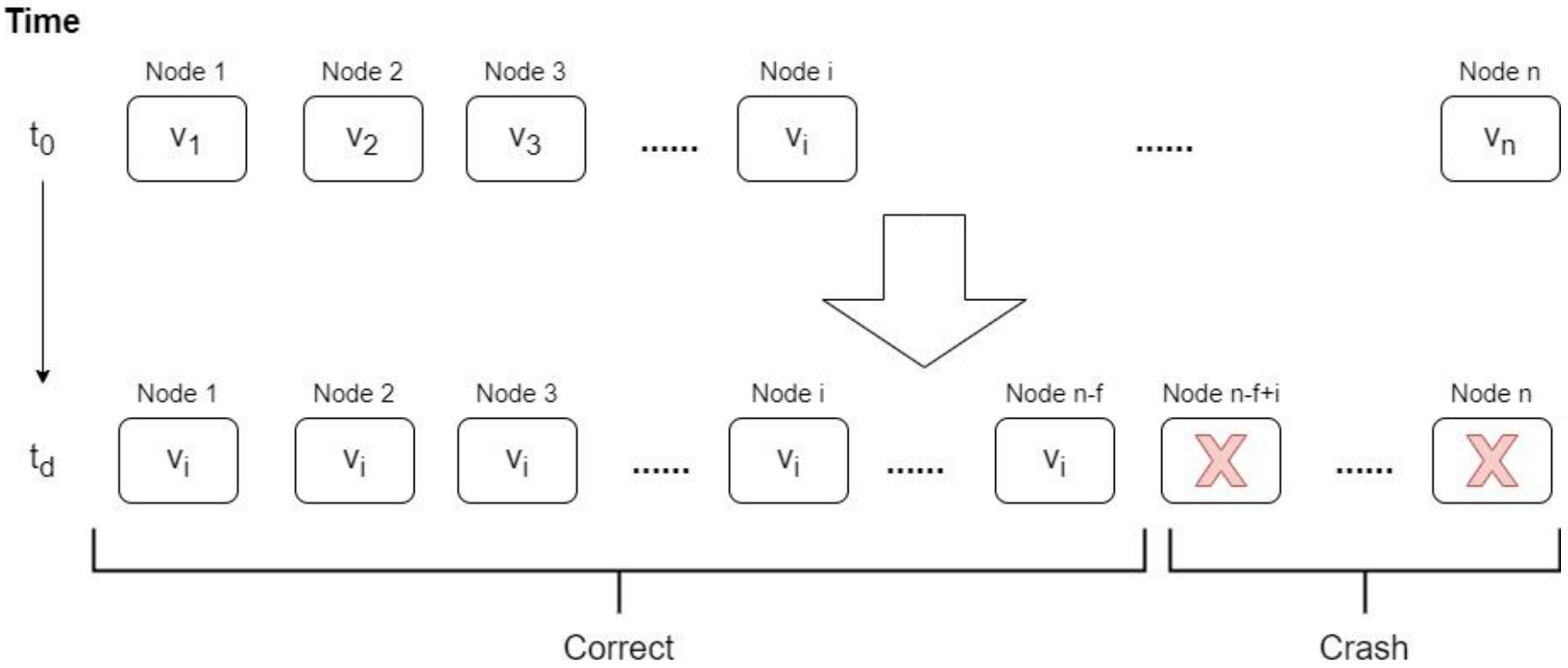
Consensus

(Definition 3.1) Consensus

There are n nodes which most f might crash.

Each node i starts with the input value v_i, the nodes must decide a value, satisfying the following 3 properties:

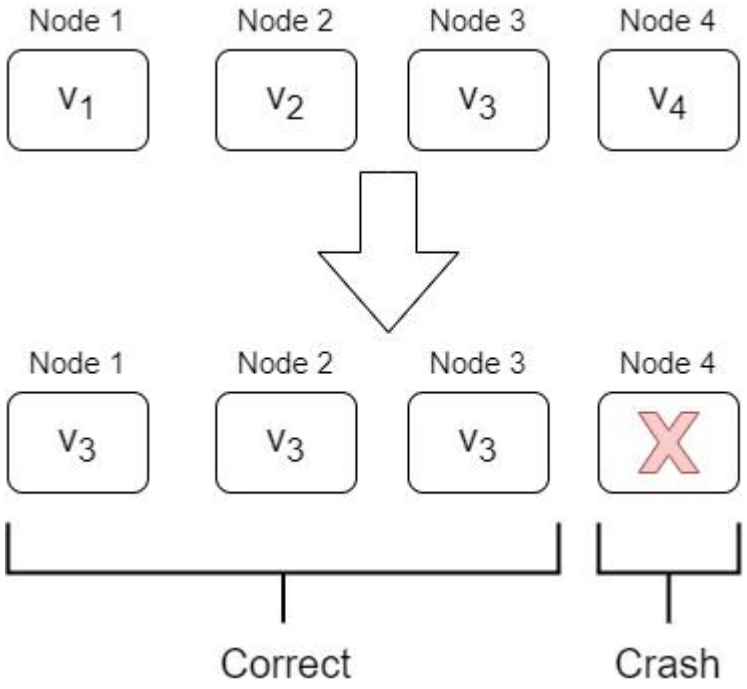
Agreement, Termination, Validity



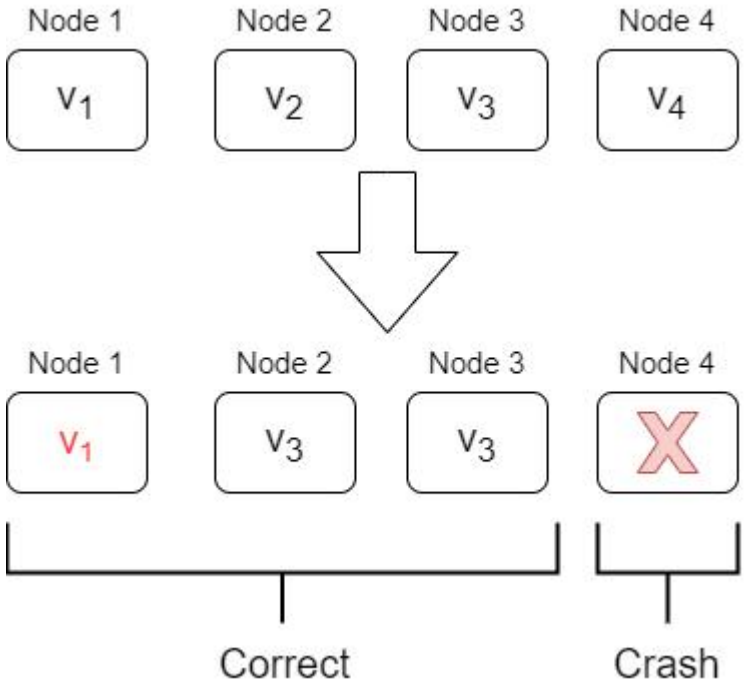
Consensus

(Definition 3.1) Consensus

1. Agreement: All correct nodes decide for **the same value**.



True: All correct nodes decide v_3 (the same value)

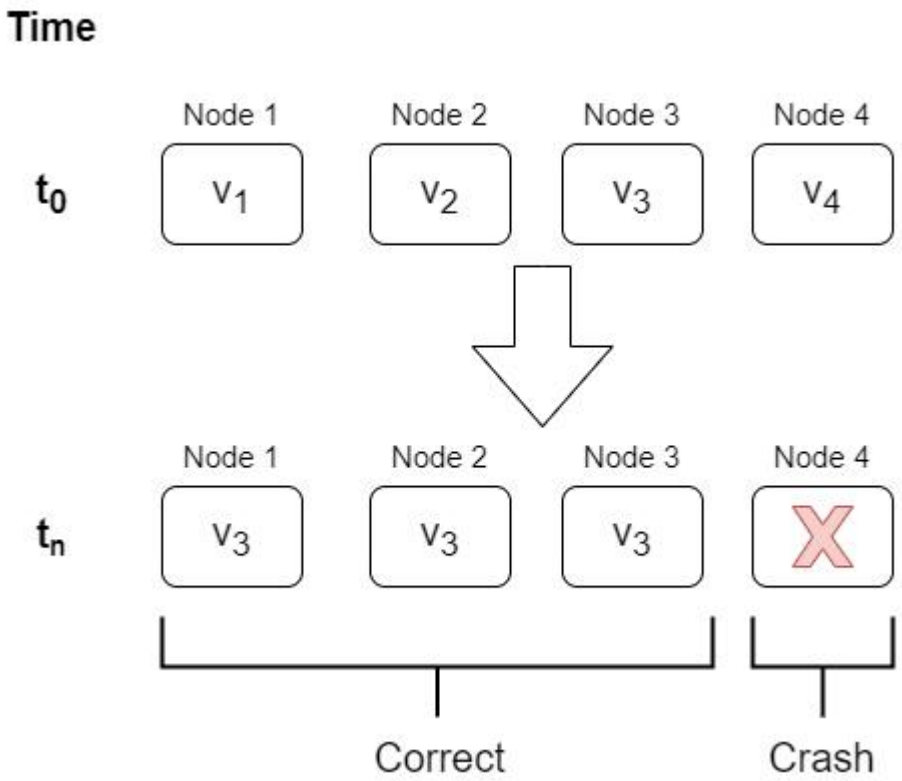


False: Some correct nodes decide v_3 and some other nodes decide v_1 (**not** the same value)

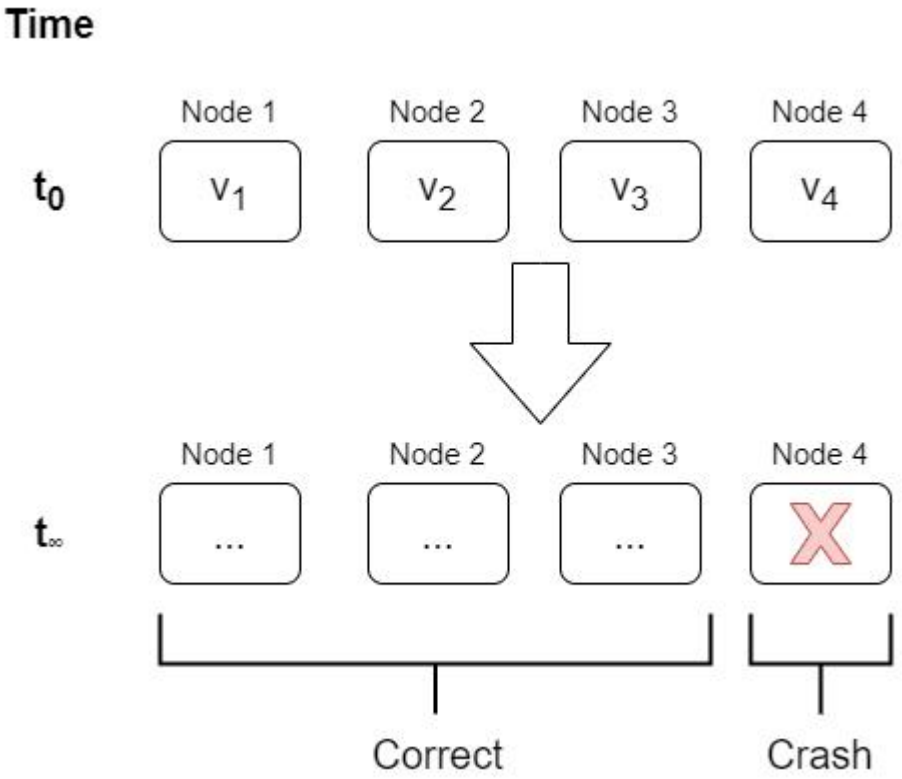
Consensus

(Definition 3.1) Consensus

2. Termination: All correct nodes terminate in **finite** time.



True: All correct nodes decide the value and terminate in finite time (t_n)

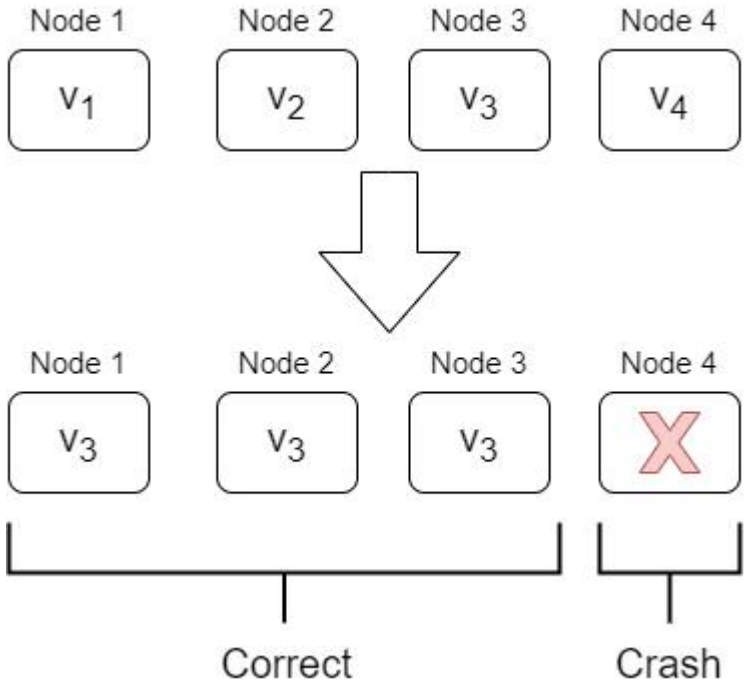


False: Some correct nodes can **not** decide a value so that they can not terminate in finite time

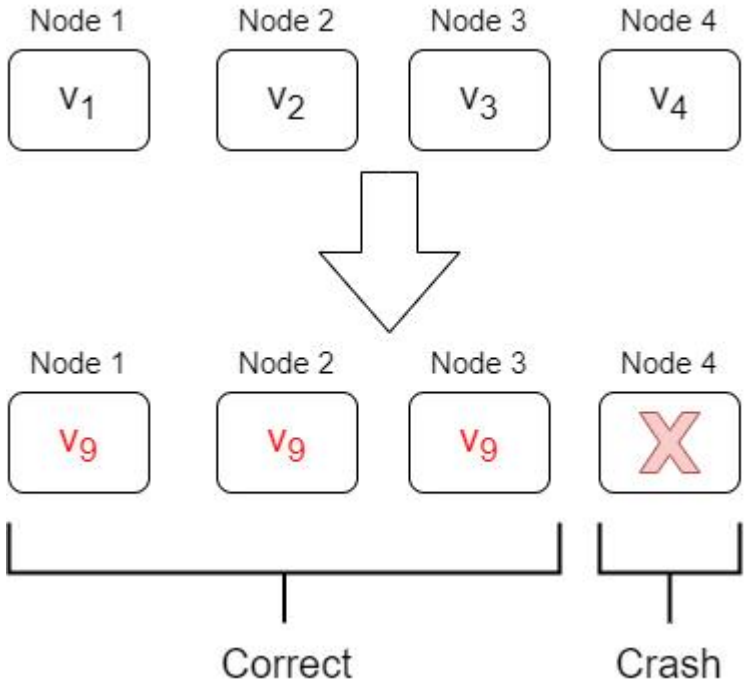
Consensus

(Definition 3.1) Consensus

3. Validity: The decision value must be the input value of a node.



True: v_3 is one of the input value of a node



False: v_9 is **not** the input value of any node

Impossibility of Consensus

Theorem 3.14

There is no deterministic algorithm which always achieves consensus in the asynchronous model, with $f > 0$.

Randomized Consensus

Theorem 3.14

There is no deterministic algorithm which always achieves consensus in the asynchronous model, with f > 0.

How to improve this situation?

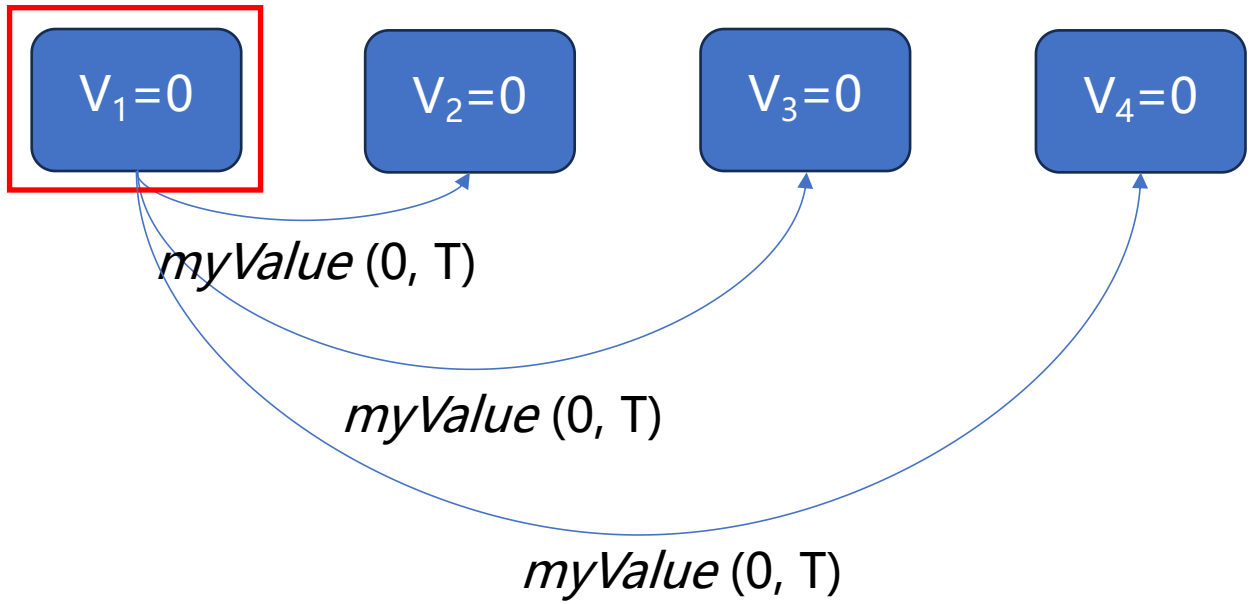
One solution is giving each node access to randomness, such as we allow each node to toss a coin.



Randomized Consensus

(Algorithm 3.15) Randomized Consensus

Example1: All v_i in same value Round = T



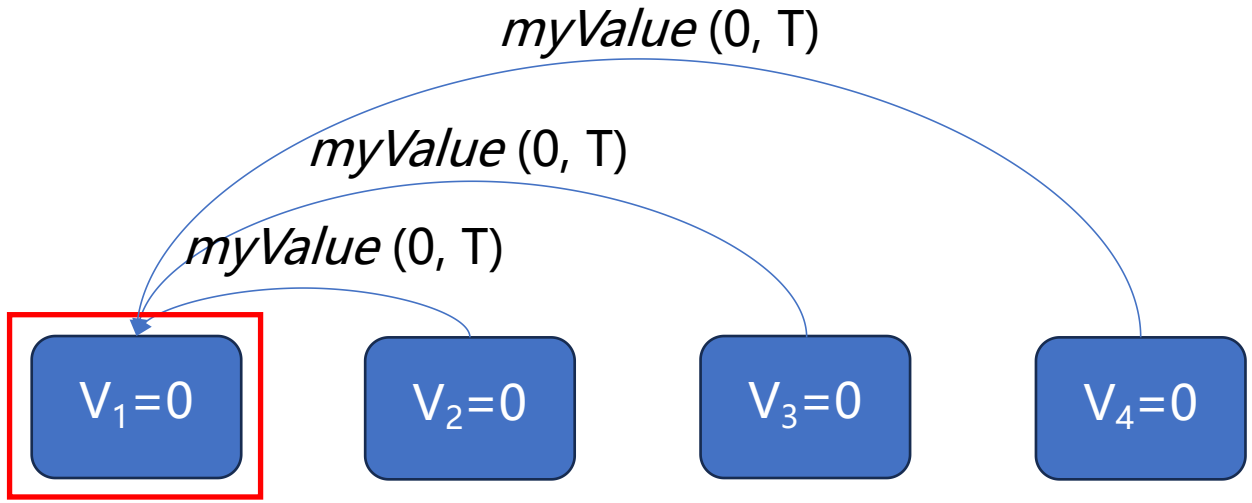
Algorithm 3.15 Randomized Consensus (assuming $f < n/2$)

```
1:  v_i ∈ {0,1}                    ◁ input bit
2:  round = 1
3:  while true do
4:    Broadcast myValue(v_i, round)
      Propose
5:    Wait until a majority of myValue messages of current round arrived
6:    if all messages contain the same value v then
7:      Broadcast propose(v, round)
8:    else
9:      Broadcast propose(⊥, round)
10:   end if
      Vote
11:   Wait until a majority of propose messages of current round arrived
12:   if all messages propose the same value v then
13:     Broadcast myValue(v, round + 1)
14:     Broadcast propose(v, round + 1)
15:     Decide for v and terminate
16:   else if there is at least one proposal for v then
17:     v_i = v
18:   else
19:     Choose v_i randomly, with Pr[v_i = 0] = Pr[v_i = 1] = 1/2
20:   end if
21:   round = round + 1
22: end while
```

Randomized Consensus

(Algorithm 3.15) Randomized Consensus

Example1: All v_i in same value Round = T



Line 6:
If -> True

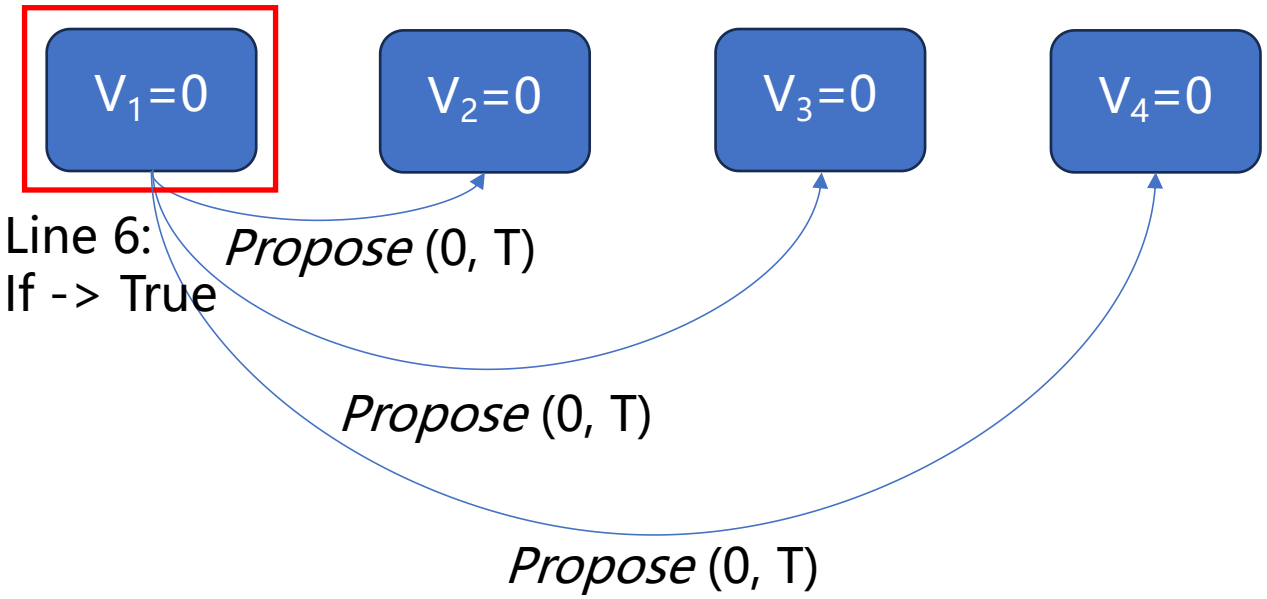
Algorithm 3.15 Randomized Consensus (assuming $f < n/2$)

```
1:  v_i ∈ {0,1}                    ◁ input bit
2:  round = 1
3:  while true do
4:    Broadcast myValue(v_i, round)
      Propose
5:    Wait until a majority of myValue messages of current round arrived
6:    if all messages contain the same value v then
7:      Broadcast propose(v, round)
8:    else
9:      Broadcast propose(⊥, round)
10:   end if
      Vote
11:   Wait until a majority of propose messages of current round arrived
12:   if all messages propose the same value v then
13:     Broadcast myValue(v, round + 1)
14:     Broadcast propose(v, round + 1)
15:     Decide for v and terminate
16:   else if there is at least one proposal for v then
17:     v_i = v
18:   else
19:     Choose v_i randomly, with Pr[v_i = 0] = Pr[v_i = 1] = 1/2
20:   end if
21:   round = round + 1
22: end while
```

Randomized Consensus

(Algorithm 3.15) Randomized Consensus

Example1: All v_i in same value Round = T



Algorithm 3.15 Randomized Consensus (assuming $f < n/2$)

```
1:  v_i ∈ {0,1}                    ◁ input bit
2:  round = 1
3:  while true do
4:    Broadcast myValue(v_i, round)

      Propose

5:    Wait until a majority of myValue messages of current round arrived
6:    if all messages contain the same value v then
7:      Broadcast propose(v, round)
8:    else
9:      Broadcast propose(⊥, round)
10:   end if

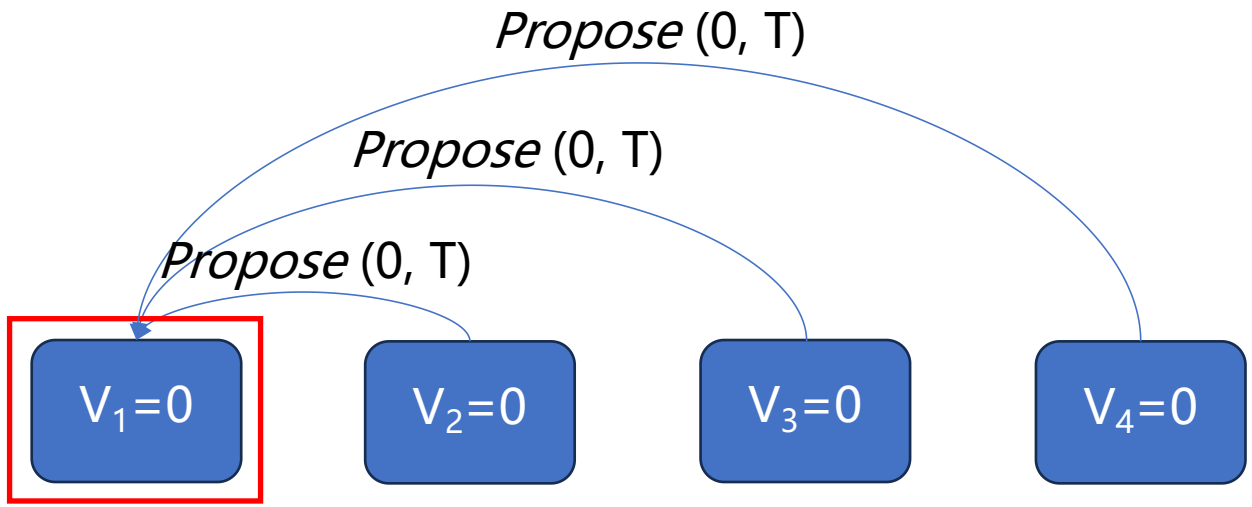
      Vote

11:   Wait until a majority of propose messages of current round arrived
12:   if all messages propose the same value v then
13:     Broadcast myValue(v, round + 1)
14:     Broadcast propose(v, round + 1)
15:     Decide for v and terminate
16:   else if there is at least one proposal for v then
17:     v_i = v
18:   else
19:     Choose v_i randomly, with Pr[v_i = 0] = Pr[v_i = 1] = 1/2
20:   end if
21:   round = round + 1
22: end while
```

Randomized Consensus

(Algorithm 3.15) Randomized Consensus

Example1: All v_i in same value Round = T



Line 12:
If -> case 1

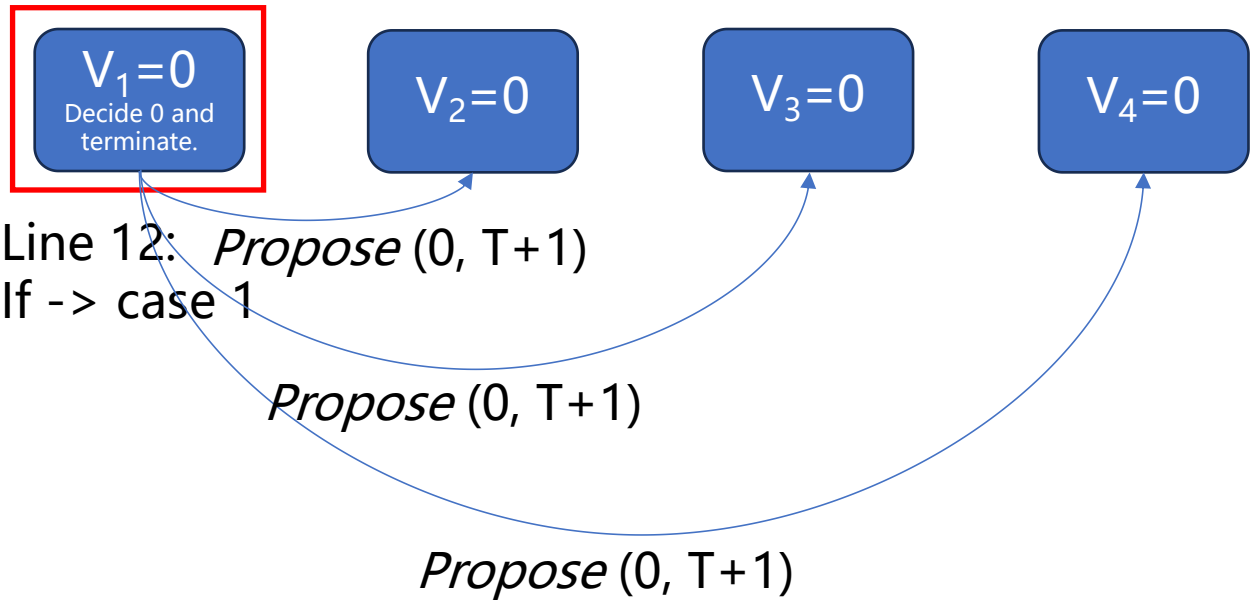
Algorithm 3.15 Randomized Consensus (assuming $f < n/2$)

```
1:  v_i ∈ {0,1}                    ◁ input bit
2:  round = 1
3:  while true do
4:    Broadcast myValue(v_i, round)
      Propose
5:    Wait until a majority of myValue messages of current round arrived
6:    if all messages contain the same value v then
7:      Broadcast propose(v, round)
8:    else
9:      Broadcast propose(⊥, round)
10:   end if
      Vote
11:   Wait until a majority of propose messages of current round arrived
12:   if all messages propose the same value v then
13: 1   Broadcast myValue(v, round + 1)
14:     Broadcast propose(v, round + 1)
15:     Decide for v and terminate
16:   else if there is at least one proposal for v then
17: 2   v_i = v
18:   else
19: 3   Choose v_i randomly, with Pr[v_i = 0] = Pr[v_i = 1] = 1/2
20:   end if
21:   round = round + 1
22: end while
```

Randomized Consensus

(Algorithm 3.15) Randomized Consensus

Example1: All v_i in same value Round = T -> T+1



Algorithm 3.15 Randomized Consensus (assuming $f < n/2$)

```
1:  v_i ∈ {0,1}                          ◁ input bit
2:  round = 1
3:  while true do
4:    Broadcast myValue(v_i, round)

      Propose

5:    Wait until a majority of myValue messages of current round arrived
6:    if all messages contain the same value v then
7:      Broadcast propose(v, round)
8:    else
9:      Broadcast propose(⊥, round)
10:   end if

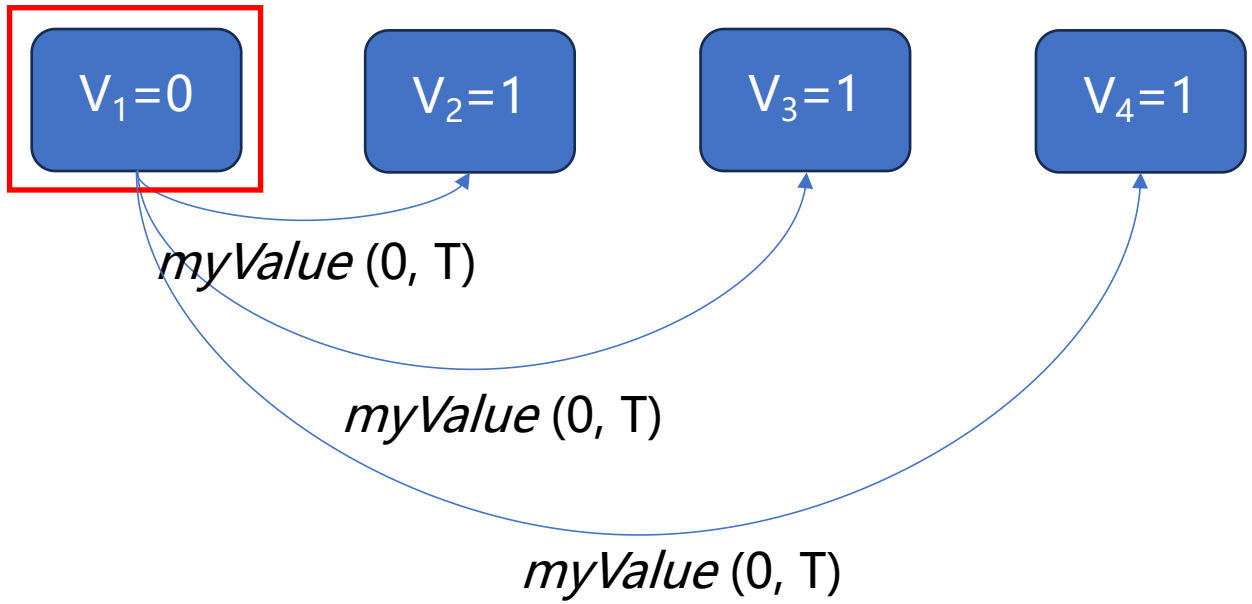
      Vote

11:   Wait until a majority of propose messages of current round arrived
12:   if all messages propose the same value v then
13: 1   Broadcast myValue(v, round + 1)
14:     Broadcast propose(v, round + 1)
15:     Decide for v and terminate
16:   else if there is at least one proposal for v then
17: 2   v_i = v
18:   else
19: 3   Choose v_i randomly, with Pr[v_i = 0] = Pr[v_i = 1] = 1/2
20:   end if
21:   round = round + 1
22: end while
```

Randomized Consensus

(Algorithm 3.15) Randomized Consensus

Example2: Normal Case Round = T



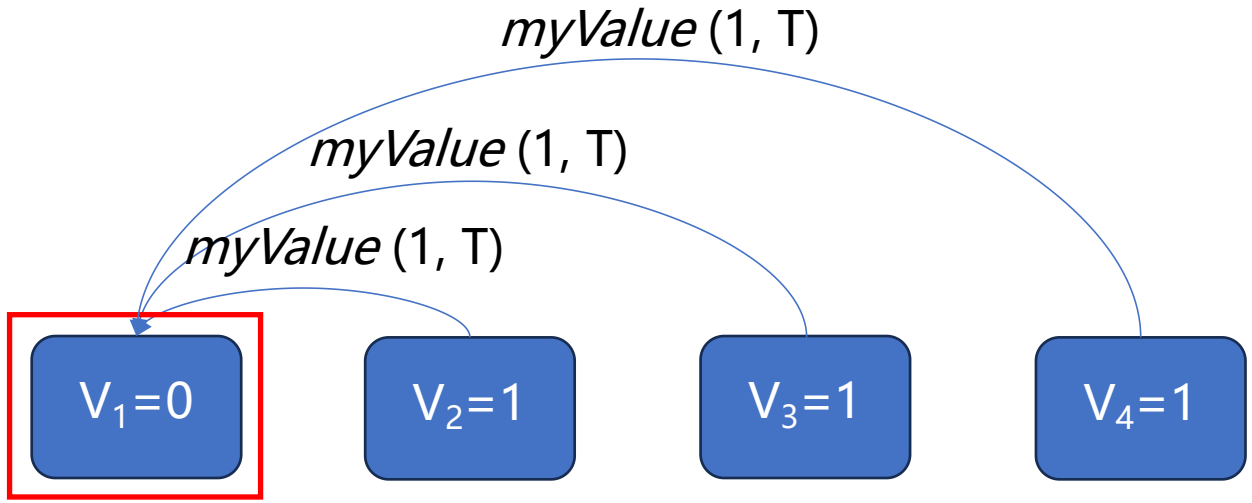
Algorithm 3.15 Randomized Consensus (assuming $f < n/2$)

```
1:  v_i ∈ {0,1}                    ◁ input bit
2:  round = 1
3:  while true do
4:    Broadcast myValue(v_i, round)
      Propose
5:    Wait until a majority of myValue messages of current round arrived
6:    if all messages contain the same value v then
7:      Broadcast propose(v, round)
8:    else
9:      Broadcast propose(⊥, round)
10:   end if
      Vote
11:   Wait until a majority of propose messages of current round arrived
12:   if all messages propose the same value v then
13:     Broadcast myValue(v, round + 1)
14:     Broadcast propose(v, round + 1)
15:     Decide for v and terminate
16:   else if there is at least one proposal for v then
17:     v_i = v
18:   else
19:     Choose v_i randomly, with Pr[v_i = 0] = Pr[v_i = 1] = 1/2
20:   end if
21:   round = round + 1
22: end while
```

Randomized Consensus

(Algorithm 3.15) Randomized Consensus

Example2: Normal Case Round = T



Line 6:
If -> False

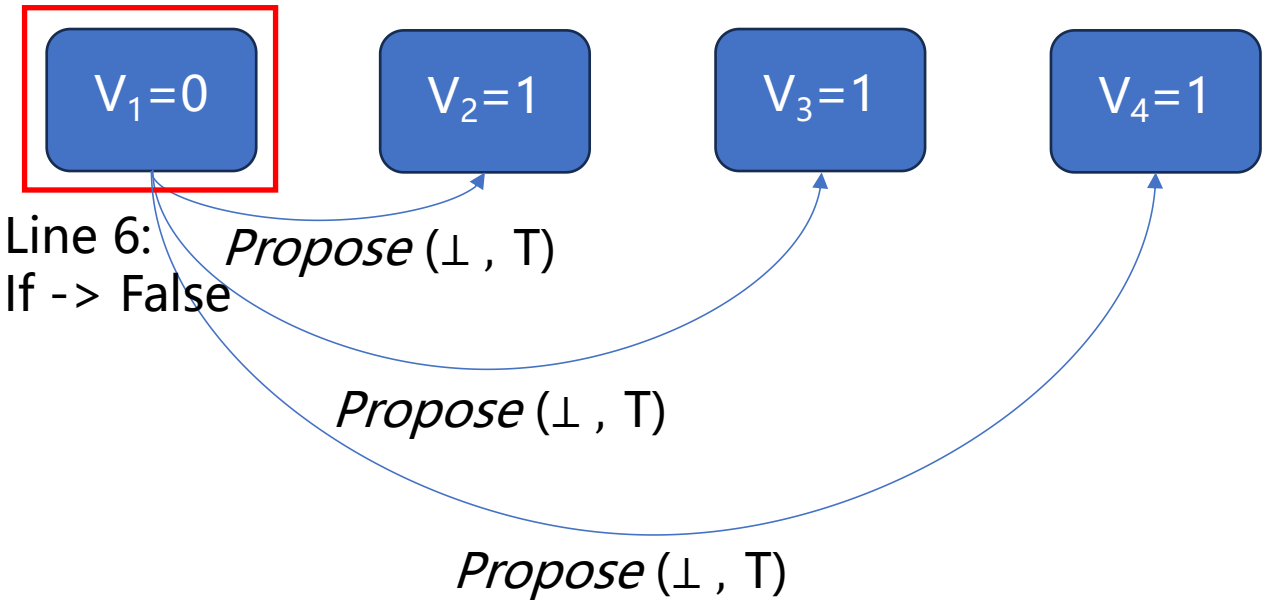
Algorithm 3.15 Randomized Consensus (assuming $f < n/2$)

```
1:  v_i ∈ {0,1}                          ◁ input bit
2:  round = 1
3:  while true do
4:    Broadcast myValue(v_i, round)
      Propose
5:    Wait until a majority of myValue messages of current round arrived
6:    if all messages contain the same value v then
7:      Broadcast propose(v, round)
8:    else
9:      Broadcast propose(⊥, round)
10:   end if
      Vote
11:   Wait until a majority of propose messages of current round arrived
12:   if all messages propose the same value v then
13:     Broadcast myValue(v, round + 1)
14:     Broadcast propose(v, round + 1)
15:     Decide for v and terminate
16:   else if there is at least one proposal for v then
17:     v_i = v
18:   else
19:     Choose v_i randomly, with Pr[v_i = 0] = Pr[v_i = 1] = 1/2
20:   end if
21:   round = round + 1
22: end while
```

Randomized Consensus

(Algorithm 3.15) Randomized Consensus

Example2: Normal Case Round = T



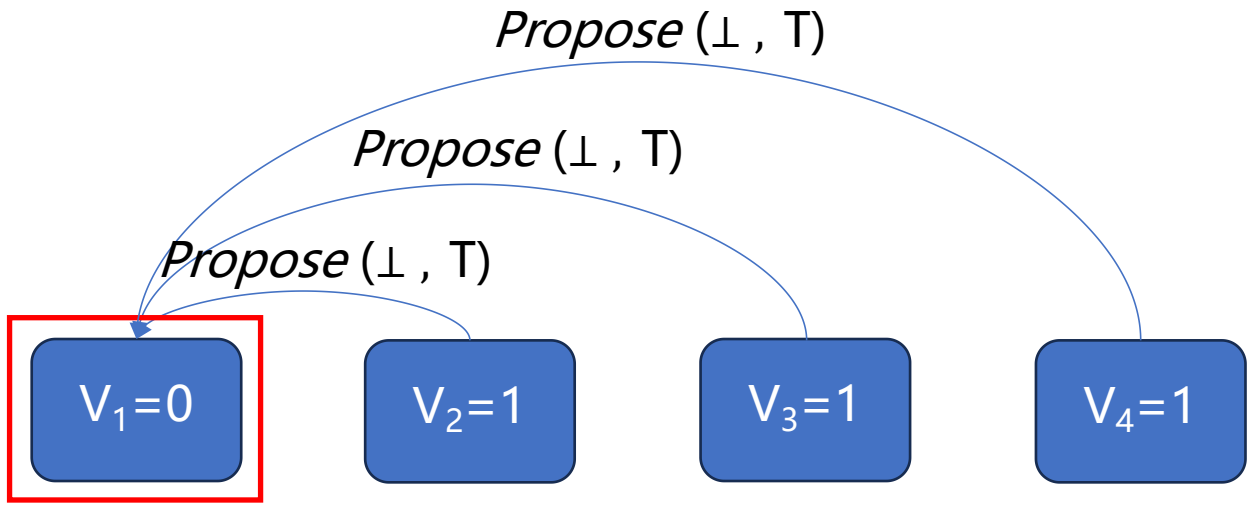
Algorithm 3.15 Randomized Consensus (assuming $f < n/2$)

```
1:  v_i ∈ {0,1}                    ◁ input bit
2:  round = 1
3:  while true do
4:    Broadcast myValue(v_i, round)
      Propose
5:    Wait until a majority of myValue messages of current round arrived
6:    if all messages contain the same value v then
7:      Broadcast propose(v, round)
8:    else
9:      Broadcast propose(⊥, round)
10:   end if
      Vote
11:   Wait until a majority of propose messages of current round arrived
12:   if all messages propose the same value v then
13:     Broadcast myValue(v, round + 1)
14:     Broadcast propose(v, round + 1)
15:     Decide for v and terminate
16:   else if there is at least one proposal for v then
17:     v_i = v
18:   else
19:     Choose v_i randomly, with Pr[v_i = 0] = Pr[v_i = 1] = 1/2
20:   end if
21:   round = round + 1
22: end while
```

Randomized Consensus

(Algorithm 3.15) Randomized Consensus

Example2: Normal Case Round = T



Line 12:
If -> case 3

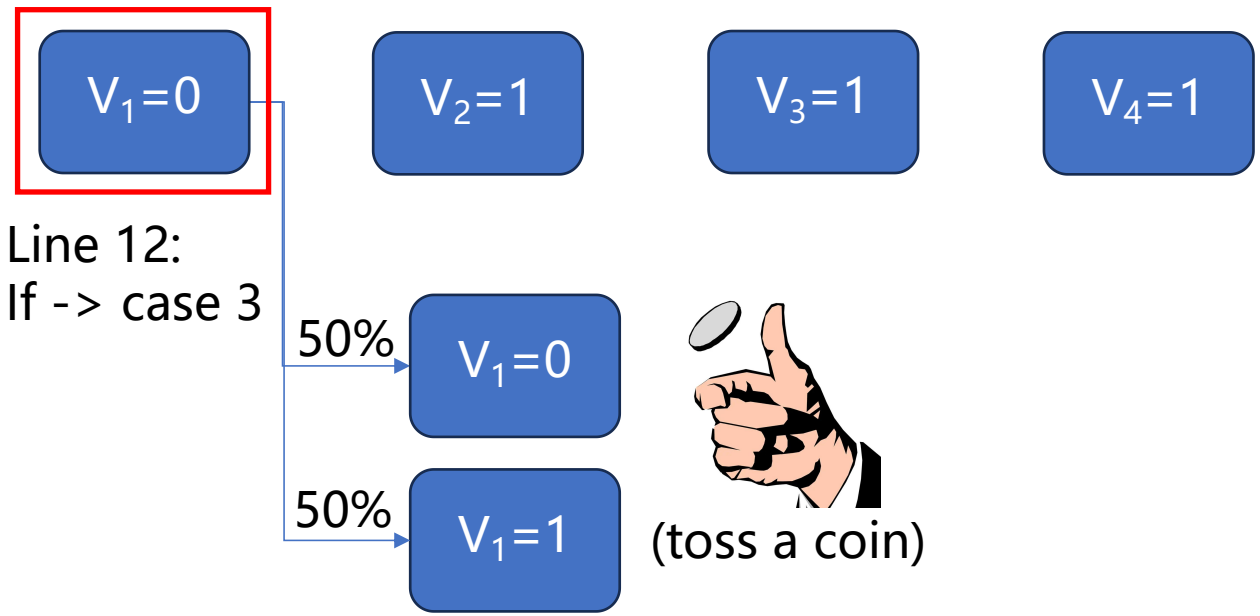
Algorithm 3.15 Randomized Consensus (assuming $f < n/2$)

```
1:  v_i ∈ {0,1}                          ◁ input bit
2:  round = 1
3:  while true do
4:    Broadcast myValue(v_i, round)
      Propose
5:    Wait until a majority of myValue messages of current round arrived
6:    if all messages contain the same value v then
7:      Broadcast propose(v, round)
8:    else
9:      Broadcast propose(⊥, round)
10:   end if
      Vote
11:   Wait until a majority of propose messages of current round arrived
12:   if all messages propose the same value v then
13: 1   Broadcast myValue(v, round + 1)
14:     Broadcast propose(v, round + 1)
15:     Decide for v and terminate
16:   else if there is at least one proposal for v then
17: 2   v_i = v
18:   else
19: 3   Choose v_i randomly, with Pr[v_i = 0] = Pr[v_i = 1] = 1/2
20:   end if
21:   round = round + 1
22: end while
```

Randomized Consensus

(Algorithm 3.15) Randomized Consensus

Example2: Normal Case Round = T -> T+1



Algorithm 3.15 Randomized Consensus (assuming $f < n/2$)

```
1:  v_i ∈ {0,1}                          ◁ input bit
2:  round = 1
3:  while true do
4:    Broadcast myValue(v_i, round)

      Propose

5:    Wait until a majority of myValue messages of current round arrived
6:    if all messages contain the same value v then
7:      Broadcast propose(v, round)
8:    else
9:      Broadcast propose(⊥, round)
10:   end if

      Vote

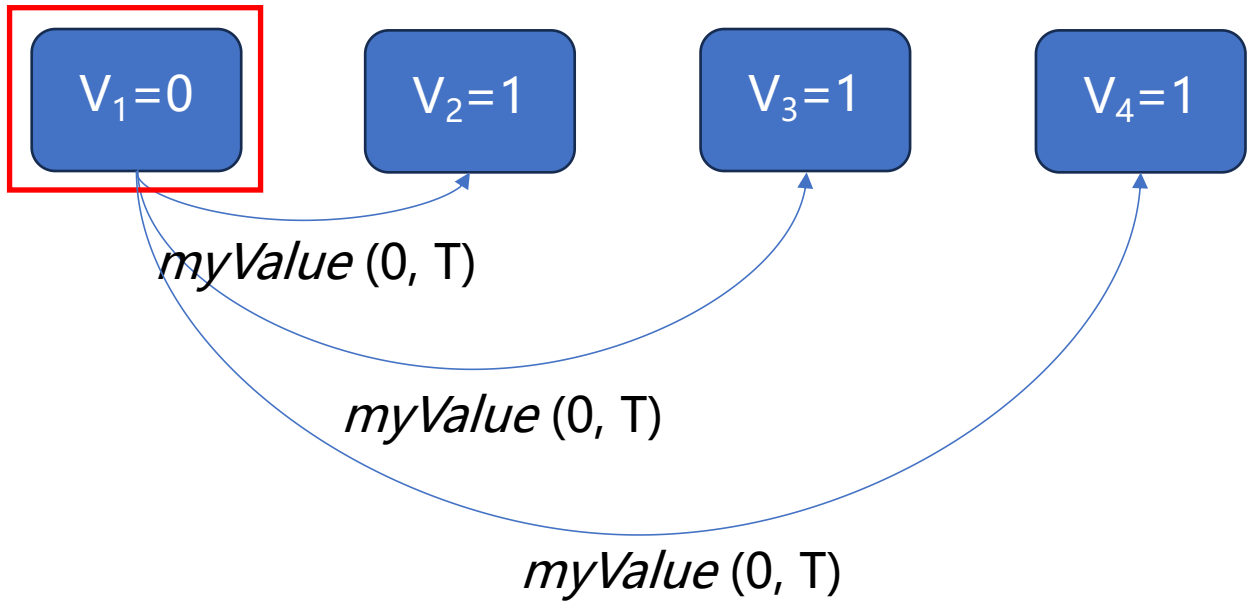
11:   Wait until a majority of propose messages of current round arrived
12:   if all messages propose the same value v then
13: 1   Broadcast myValue(v, round + 1)
14:     Broadcast propose(v, round + 1)
15:     Decide for v and terminate
16:   else if there is at least one proposal for v then
17: 2   v_i = v
18:   else
19: 3   Choose v_i randomly, with Pr[v_i = 0] = Pr[v_i = 1] = 1/2
20:   end if
21:   round = round + 1
22: end while
```

Randomized Consensus

(Algorithm 3.15) Randomized Consensus

Example3:
Some messages delay.

Round = T



Algorithm 3.15 Randomized Consensus (assuming $f < n/2$)

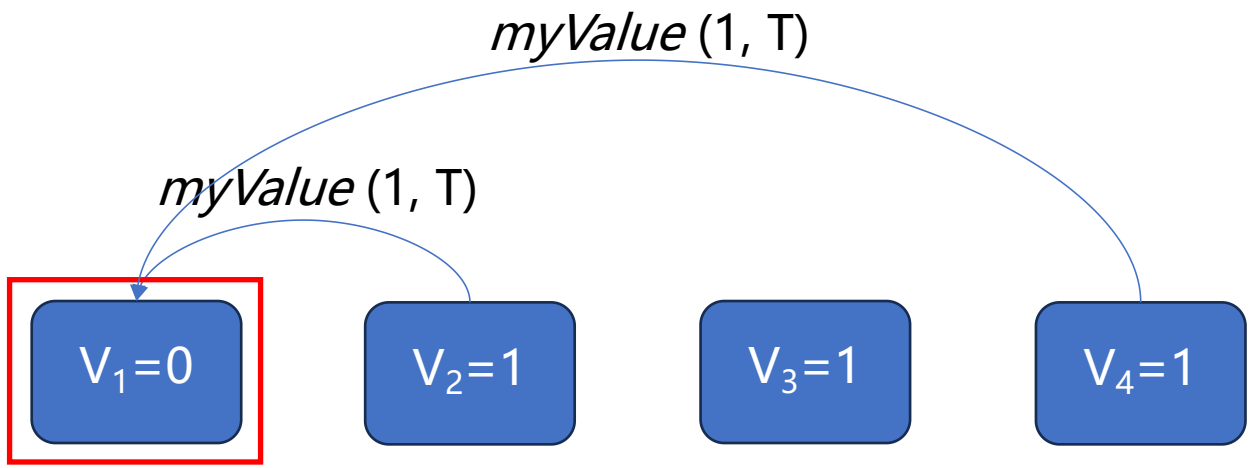
```
1:  v_i ∈ {0,1}                          ◁ input bit
2:  round = 1
3:  while true do
4:    Broadcast myValue(v_i, round)
      Propose
5:    Wait until a majority of myValue messages of current round arrived
6:    if all messages contain the same value v then
7:      Broadcast propose(v, round)
8:    else
9:      Broadcast propose(⊥, round)
10:   end if
      Vote
11:   Wait until a majority of propose messages of current round arrived
12:   if all messages propose the same value v then
13:     Broadcast myValue(v, round + 1)
14:     Broadcast propose(v, round + 1)
15:     Decide for v and terminate
16:   else if there is at least one proposal for v then
17:     v_i = v
18:   else
19:     Choose v_i randomly, with Pr[v_i = 0] = Pr[v_i = 1] = 1/2
20:   end if
21:   round = round + 1
22: end while
```

Randomized Consensus

(Algorithm 3.15) Randomized Consensus

Example3:
Some messages delay.

Round = T



Line 6:
If -> False

Algorithm 3.15 Randomized Consensus (assuming $f < n/2$)

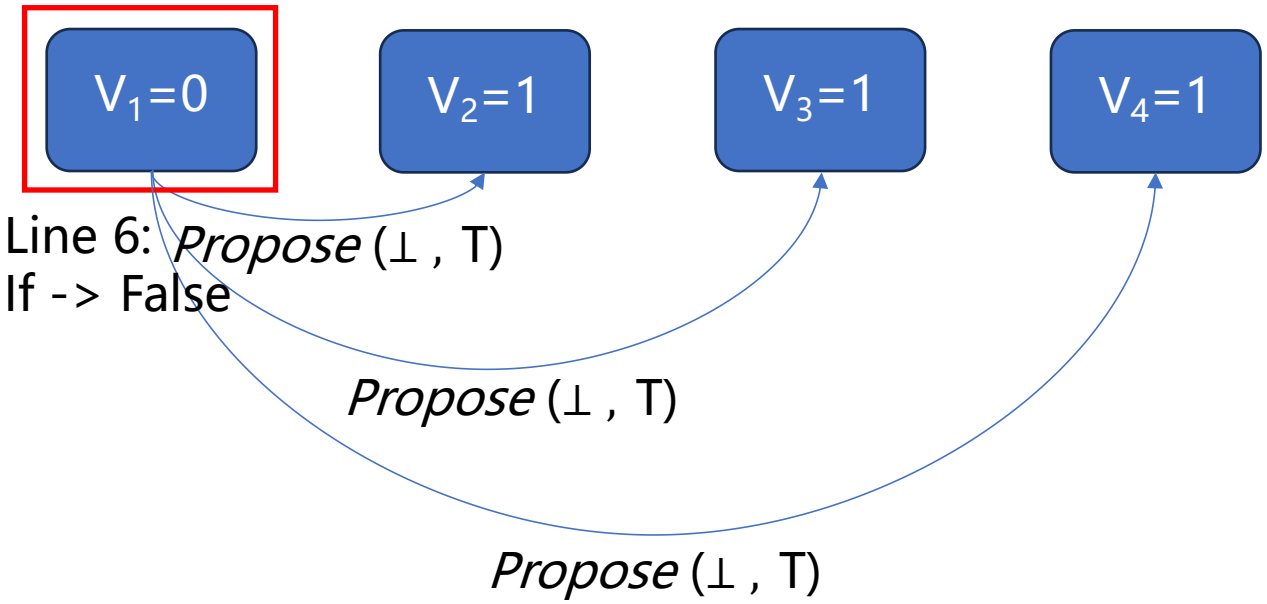
```
1:  v_i ∈ {0,1}                          ◁ input bit
2:  round = 1
3:  while true do
4:    Broadcast myValue(v_i, round)
      Propose
5:    Wait until a majority of myValue messages of current round arrived
6:    if all messages contain the same value v then
7:      Broadcast propose(v, round)
8:    else
9:      Broadcast propose(⊥, round)
10:   end if
      Vote
11:   Wait until a majority of propose messages of current round arrived
12:   if all messages propose the same value v then
13:     Broadcast myValue(v, round + 1)
14:     Broadcast propose(v, round + 1)
15:     Decide for v and terminate
16:   else if there is at least one proposal for v then
17:     v_i = v
18:   else
19:     Choose v_i randomly, with Pr[v_i = 0] = Pr[v_i = 1] = 1/2
20:   end if
21:   round = round + 1
22: end while
```

Randomized Consensus

(Algorithm 3.15) Randomized Consensus

Example3:
Some messages delay.

Round = T



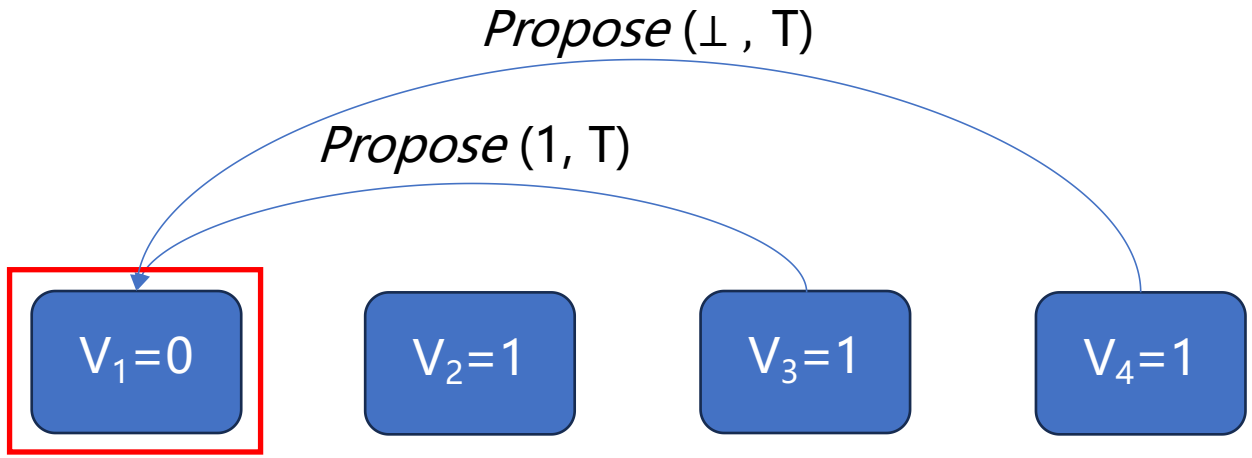
Algorithm 3.15 Randomized Consensus (assuming $f < n/2$)

```
1:  v_i ∈ {0,1}                          ◁ input bit
2:  round = 1
3:  while true do
4:    Broadcast myValue(v_i, round)
      Propose
5:    Wait until a majority of myValue messages of current round arrived
6:    if all messages contain the same value v then
7:      Broadcast propose(v, round)
8:    else
9:      Broadcast propose(⊥, round)
10:   end if
      Vote
11:   Wait until a majority of propose messages of current round arrived
12:   if all messages propose the same value v then
13:     Broadcast myValue(v, round + 1)
14:     Broadcast propose(v, round + 1)
15:     Decide for v and terminate
16:   else if there is at least one proposal for v then
17:     v_i = v
18:   else
19:     Choose v_i randomly, with Pr[v_i = 0] = Pr[v_i = 1] = 1/2
20:   end if
21:   round = round + 1
22: end while
```

Randomized Consensus

(Algorithm 3.15) Randomized Consensus

Example3: Round = T
Some messages delay.



Line 12:
If -> case 2

Algorithm 3.15 Randomized Consensus (assuming $f < n/2$)

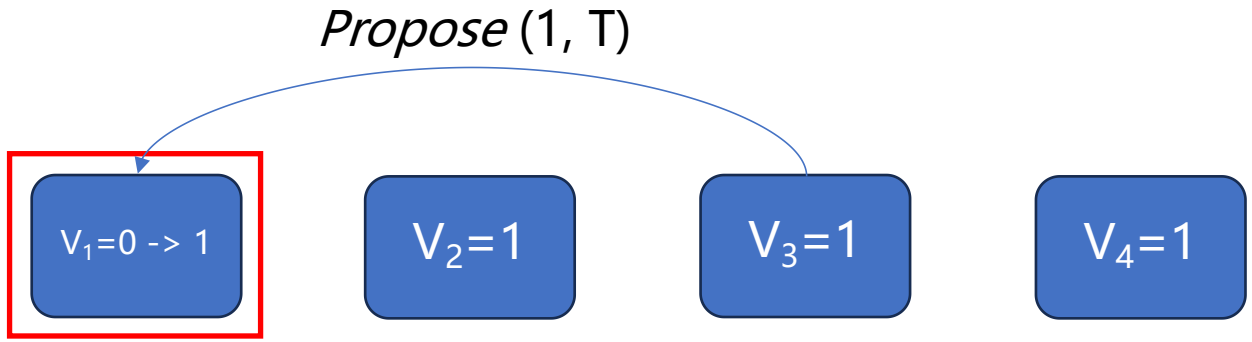
```
1:  v_i ∈ {0,1}                          ◁ input bit
2:  round = 1
3:  while true do
4:    Broadcast myValue(v_i, round)
      Propose
5:    Wait until a majority of myValue messages of current round arrived
6:    if all messages contain the same value v then
7:      Broadcast propose(v, round)
8:    else
9:      Broadcast propose(⊥, round)
10:   end if
      Vote
11:   Wait until a majority of propose messages of current round arrived
12:   if all messages propose the same value v then
13: 1   Broadcast myValue(v, round + 1)
14:     Broadcast propose(v, round + 1)
15:     Decide for v and terminate
16:   else if there is at least one proposal for v then
17: 2   v_i = v
18:   else
19: 3   Choose v_i randomly, with Pr[v_i = 0] = Pr[v_i = 1] = 1/2
20:   end if
21:   round = round + 1
22: end while
```

Randomized Consensus

(Algorithm 3.15) Randomized Consensus

Example3:
Some messages delay.

Round = T -> T+1



Line 12:
If -> case 2

Algorithm 3.15 Randomized Consensus (assuming $f < n/2$)

```
1:  v_i ∈ {0,1}                          ◁ input bit
2:  round = 1
3:  while true do
4:    Broadcast myValue(v_i, round)
      Propose
5:    Wait until a majority of myValue messages of current round arrived
6:    if all messages contain the same value v then
7:      Broadcast propose(v, round)
8:    else
9:      Broadcast propose(⊥, round)
10:   end if
      Vote
11:   Wait until a majority of propose messages of current round arrived
12:   if all messages propose the same value v then
13: 1   Broadcast myValue(v, round + 1)
14:     Broadcast propose(v, round + 1)
15:     Decide for v and terminate
16:   else if there is at least one proposal for v then
17: 2   v_i = v
18:   else
19: 3   Choose v_i randomly, with Pr[v_i = 0] = Pr[v_i = 1] = 1/2
20:   end if
21:   round = round + 1
22: end while
```

Randomized Consensus

Lemma 3.16

As long as no node decides and terminates, Algorithm 3.15 does not get stuck, independent of which nodes crash.

Proof:

1. Node Waiting point: line 5 and line 11.
   (wait until a majority of messages arrived.)
2. Since *f < n/2* and no node decides and terminates,
   every node will always receive more than *n/2* messages.
3. So, the algorithm does not get stuck.

Algorithm 3.15 Randomized Consensus (assuming $f < n/2$)

```
1:  v_i ∈ {0,1}                    ◁ input bit
2:  round = 1
3:  while true do
4:    Broadcast myValue(v_i, round)
      Propose
5:    Wait until a majority of myValue messages of current round arrived
6:    if all messages contain the same value v then
7:      Broadcast propose(v, round)
8:    else
9:      Broadcast propose(⊥, round)
10:   end if
      Vote
11:   Wait until a majority of propose messages of current round arrived
12:   if all messages propose the same value v then
13:     Broadcast myValue(v, round + 1)
14:     Broadcast propose(v, round + 1)
15:     Decide for v and terminate
16:   else if there is at least one proposal for v then
17:     v_i = v
18:   else
19:     Choose v_i randomly, with Pr[v_i = 0] = Pr[v_i = 1] = 1/2
20:   end if
21:   round = round + 1
22: end while
```

Randomized Consensus

Lemma 3.17

Algorithm 3.15 satisfies the **validity** requirement.

Proof:

Case 1: if v_i are all 0 or all 1, all nodes can hear all same proposals in the first round. Then they decide the value and terminate in the same round, so the algorithm satisfies the validity requirement.

Case 2: If v_i includes both 0 and 1, it means both 0 and 1 are acceptable. As v_i would not be the number expect 0 and 1, the algorithm satisfies the validity requirement.

Algorithm 3.15 Randomized Consensus (assuming $f < n/2$)

```
1:  v_i ∈ {0,1}                          ◁ input bit
2:  round = 1
3:  while true do
4:    Broadcast myValue(v_i, round)
      Propose
5:    Wait until a majority of myValue messages of current round arrived
6:    if all messages contain the same value v then
7:      Broadcast propose(v, round)
8:    else
9:      Broadcast propose(⊥, round)
10:   end if
      Vote
11:   Wait until a majority of propose messages of current round arrived
12:   if all messages propose the same value v then
13:     Broadcast myValue(v, round + 1)
14:     Broadcast propose(v, round + 1)
15:     Decide for v and terminate
16:   else if there is at least one proposal for v then
17:     v_i = v
18:   else
19:     Choose v_i randomly, with Pr[v_i = 0] = Pr[v_i = 1] = 1/2
20:   end if
21:   round = round + 1
22: end while
```

Randomized Consensus

Lemma 3.18

Algorithm 3.15 satisfies the **agreement** requirement.

Proof:

1. Proposal 0 and 1 can not appear in the same round as only the majority of *myValue* can build the proposal.
2. Let u as the first node that decide *v* and terminate in round *T*. That means it received a majority proposal of *v*. Round *T* can' t decide another value.
3. Other nodes in round *T* can experience one of two scenarios:
   (1) can also receive the majority proposals. (agreement)
   (2) can' t receive the majority proposals.

Algorithm 3.15 Randomized Consensus (assuming $f < n/2$)

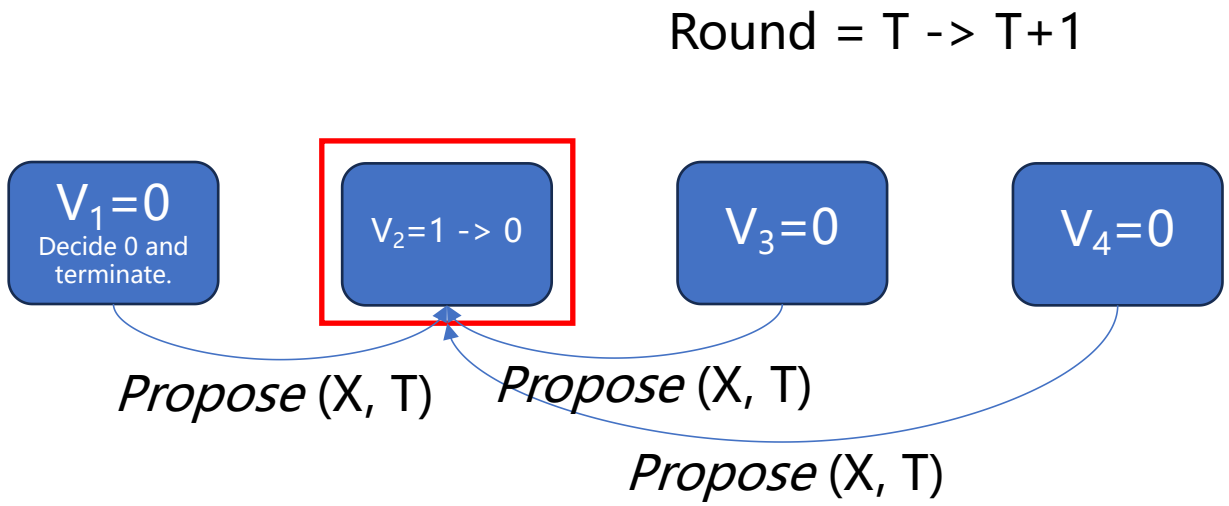
```
1:  v_i ∈ {0,1}                    ◁ input bit
2:  round = 1
3:  while true do
4:    Broadcast myValue(v_i, round)
      Propose
5:    Wait until a majority of myValue messages of current round arrived
6:    if all messages contain the same value v then
7:      Broadcast propose(v, round)
8:    else
9:      Broadcast propose(⊥, round)
10:   end if
      Vote
11:   Wait until a majority of propose messages of current round arrived
12:   if all messages propose the same value v then
13:     Broadcast myValue(v, round + 1)
14:     Broadcast propose(v, round + 1)
15:     Decide for v and terminate
16:   else if there is at least one proposal for v then
17:     v_i = v
18:   else
19:     Choose v_i randomly, with Pr[v_i = 0] = Pr[v_i = 1] = 1/2
20:   end if
21:   round = round + 1
22: end while
```

Randomized Consensus

Lemma 3.18

Algorithm 3.15 satisfies the **agreement** requirement.

Proof: 3(2)



- At least one *Propose* (0, T) as V_1 decides 0.
- Then all nodes become the same value so that all nodes can terminate in round *T+1*. (same as example 2 in Algorithm 3.15)

Algorithm 3.15 Randomized Consensus (assuming $f < n/2$)

```
1:  v_i ∈ {0,1}                          ◁ input bit
2:  round = 1
3:  while true do
4:    Broadcast myValue(v_i, round)

      Propose

5:    Wait until a majority of myValue messages of current round arrived
6:    if all messages contain the same value v then
7:      Broadcast propose(v, round)
8:    else
9:      Broadcast propose(⊥, round)
10:   end if

      Vote

11:   Wait until a majority of propose messages of current round arrived
12:   if all messages propose the same value v then
13:     Broadcast myValue(v, round + 1)
14:     Broadcast propose(v, round + 1)
15:     Decide for v and terminate
16:   else if there is at least one proposal for v then
17:     v_i = v
18:   else
19:     Choose v_i randomly, with Pr[v_i = 0] = Pr[v_i = 1] = 1/2
20:   end if
21:   round = round + 1
22: end while
```

Randomized Consensus

Lemma 3.19 and Theorem 3.20

Algorithm 3.15 satisfies the **termination** requirement.

All nodes terminate in expected time $O(2^n)$ if binary input and f < n/2.

Proof:

1. In Lemma 3.18, it shows that when the first node terminate in round *T*, all nodes terminate in at most round *T+1*.
2. Assume no node receives the majority proposal in round *T*, some nodes can update the value by proposal. The others can only choose v_i randomly(line 19). The probability of all nodes choose the same value in one round is $\frac{1}{2^n}$, the expected number of round is $O(2^n)$. As each node broadcast 2 messages in a round, the expected number of runtime is also $O(2^n)$.

Algorithm 3.15 Randomized Consensus (assuming $f < n/2$)

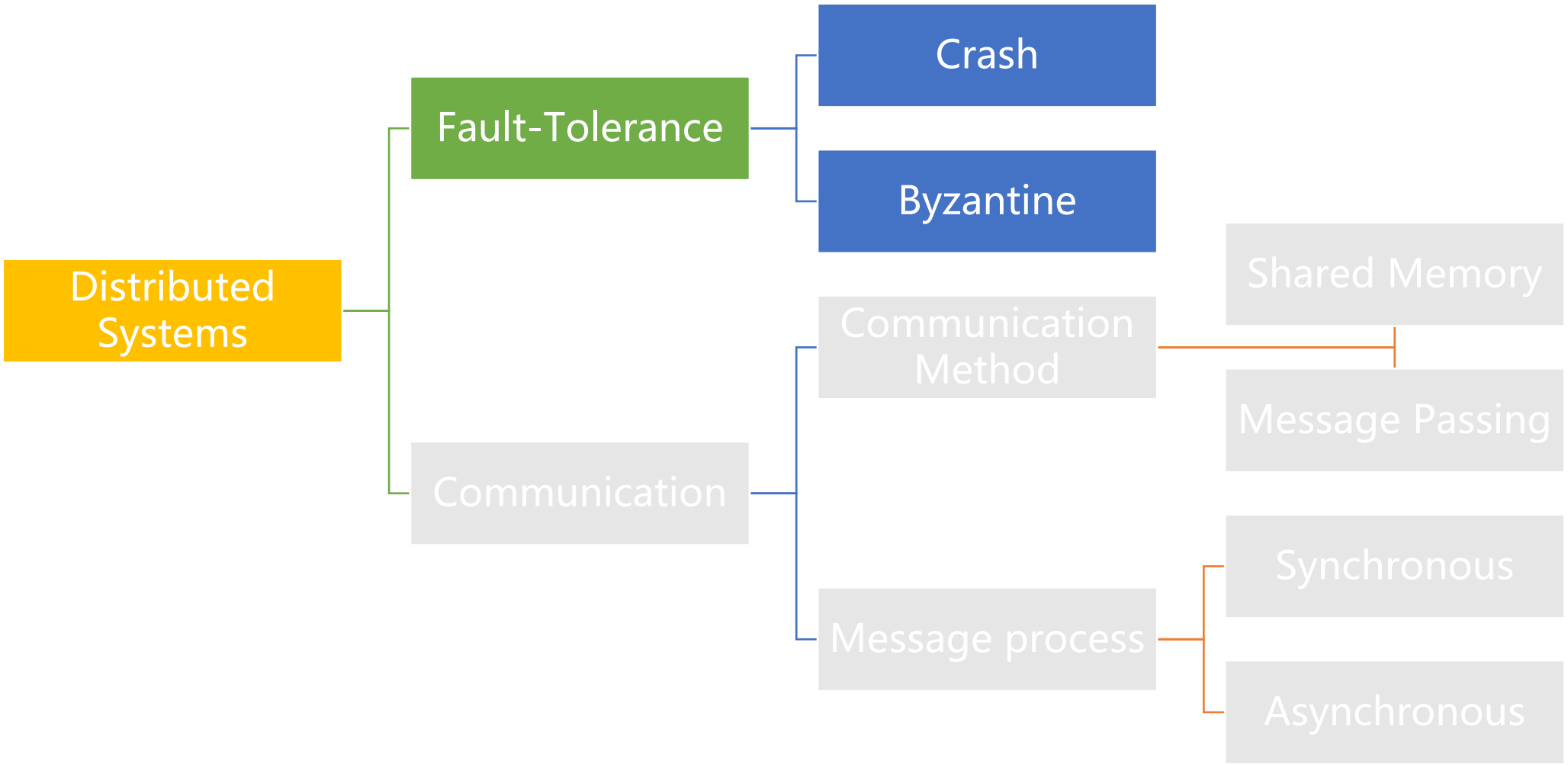
```
1:  v_i ∈ {0,1}                          ◁ input bit
2:  round = 1
3:  while true do
4:    Broadcast myValue(v_i, round)
      Propose
5:    Wait until a majority of myValue messages of current round arrived
6:    if all messages contain the same value v then
7:      Broadcast propose(v, round)
8:    else
9:      Broadcast propose(⊥, round)
10:   end if
      Vote
11:   Wait until a majority of propose messages of current round arrived
12:   if all messages propose the same value v then
13:     Broadcast myValue(v, round + 1)
14:     Broadcast propose(v, round + 1)
15:     Decide for v and terminate
16:   else if there is at least one proposal for v then
17:     v_i = v
18:   else
19:     Choose v_i randomly, with Pr[v_i = 0] = Pr[v_i = 1] = 1/2
20:   end if
21:   round = round + 1
22: end while
```

Randomized Consensus

Theorem 3.21

There is no consensus algorithm for the asynchronous model that tolerates $f \geq n/2$ many failures.

Basic Concepts of Distributed Systems



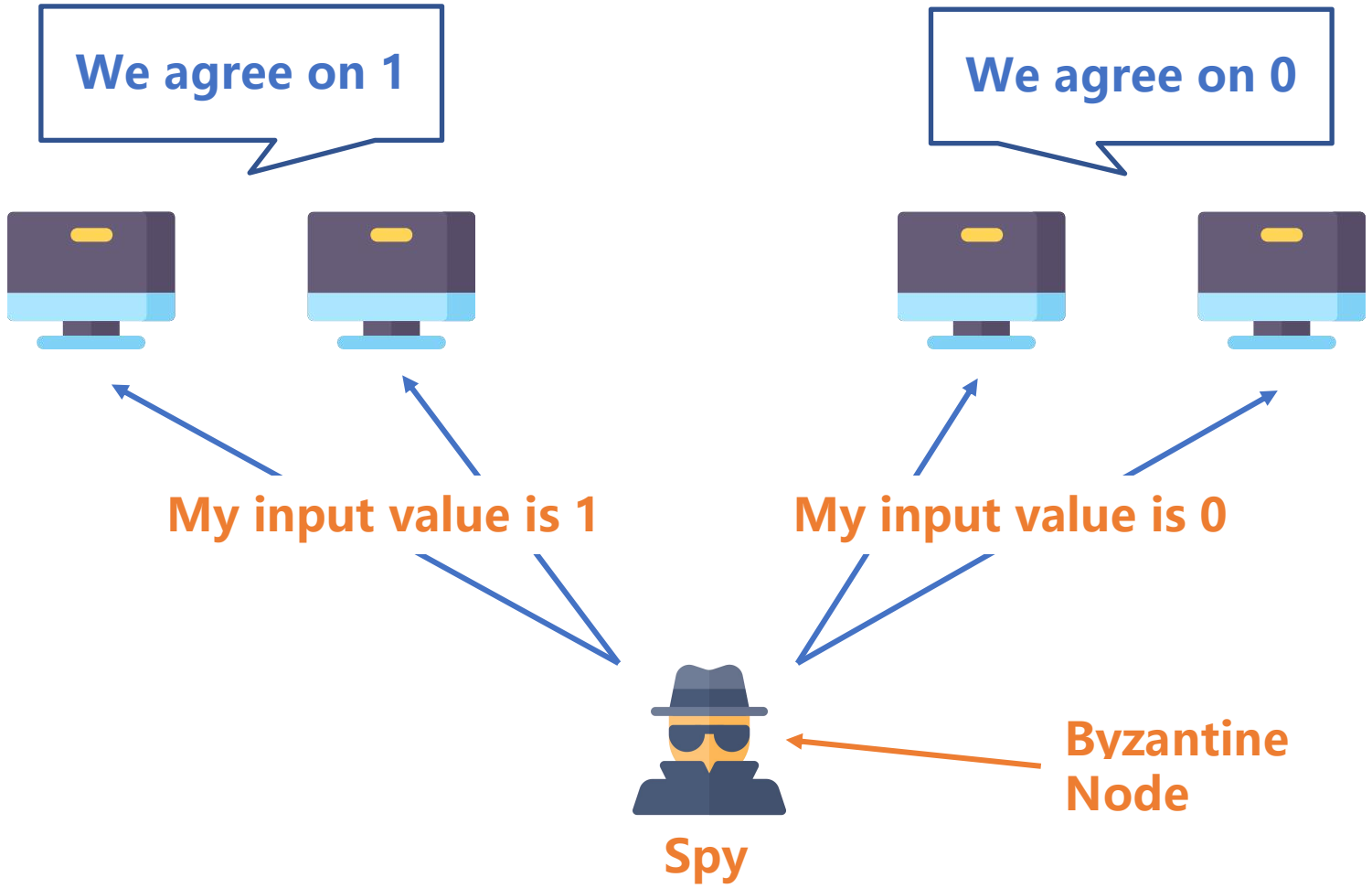
Spy in the Distributed System

- We often see spy-themed works in movies and TV shows
- They assume false identities and use disinformation to disrupt enemies
- Does such a role exist in a distributed system?



Spy in the Distributed System

- **Failed to reach consensus**



Byzantine Node

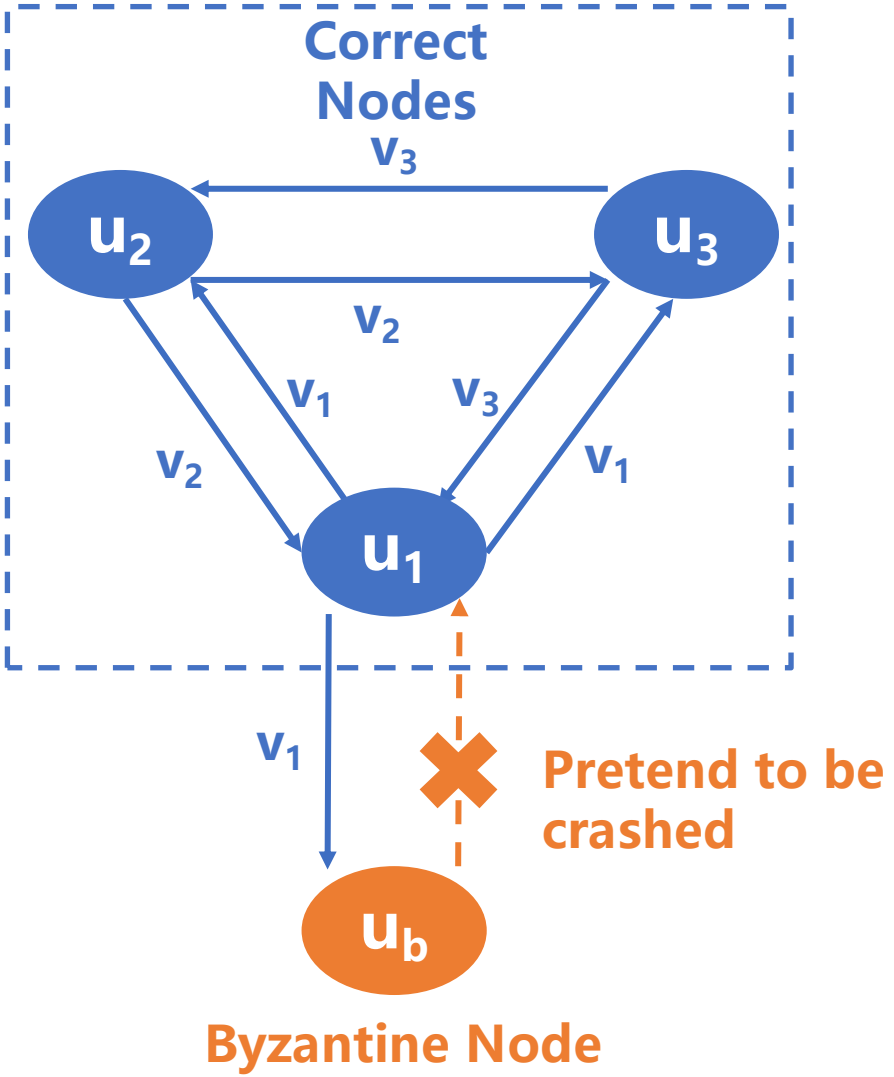
(Definition 4.1) Byzantine

A node which can have **arbitrary behavior** is called byzantine.

This includes **"anything imaginable"** :

- Not sending any messages at all;
- Sending different and wrong messages to different neighbors;
- Lying about the input value;
- Colluding with other byzantine nodes.

*We call non-byzantine nodes **correct nodes**



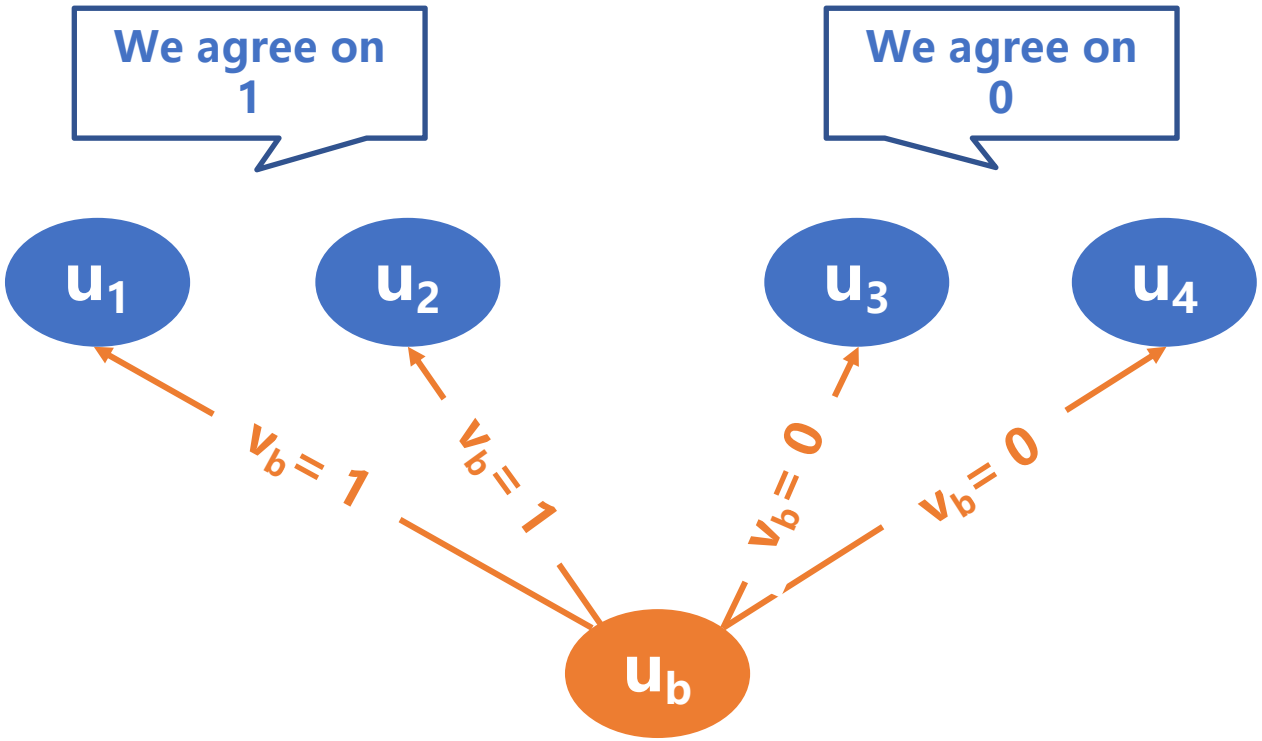
Byzantine Node

(Definition 4.1) Byzantine

A node which can have **arbitrary behavior** is called byzantine.

This includes **"anything imaginable"**:

- Not sending any messages at all;
- Sending different and wrong messages to different neighbors;
- Lying about the input value;
- Colluding with other byzantine nodes.



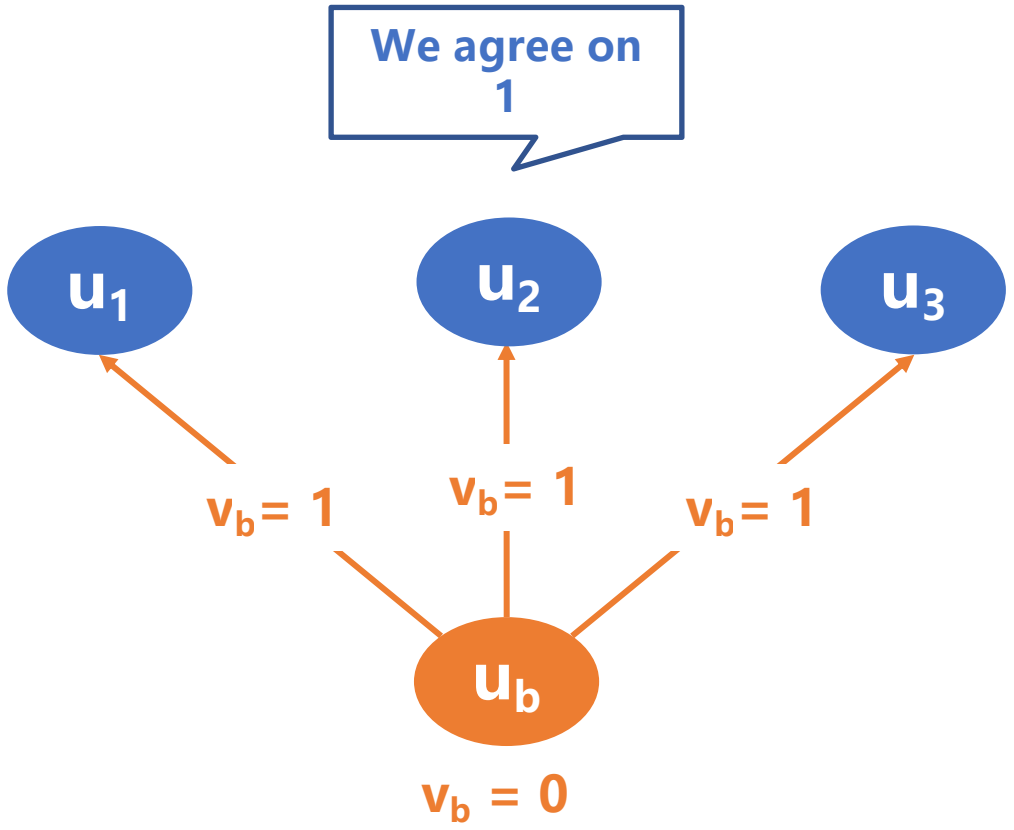
Byzantine Node

(Definition 4.1) Byzantine

A node which can have **arbitrary behavior** is called byzantine.

This includes **"anything imaginable"**:

- Not sending any messages at all;
- Sending different and wrong messages to different neighbors;
- Lying about the input value;
- Colluding with other byzantine nodes.



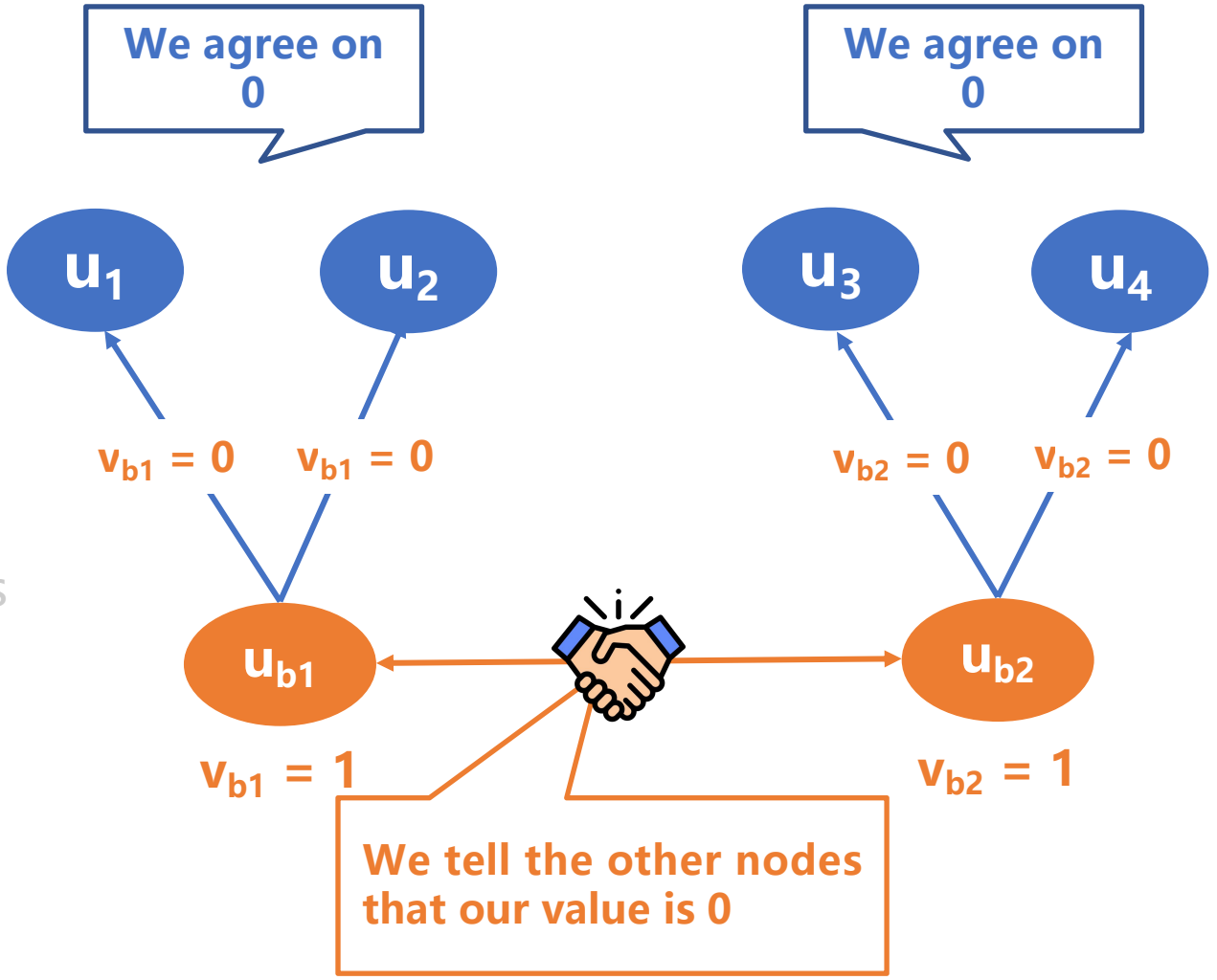
Byzantine Node

(Definition 4.1) Byzantine

A node which can have **arbitrary behavior** is called byzantine.

This includes **"anything imaginable"**:

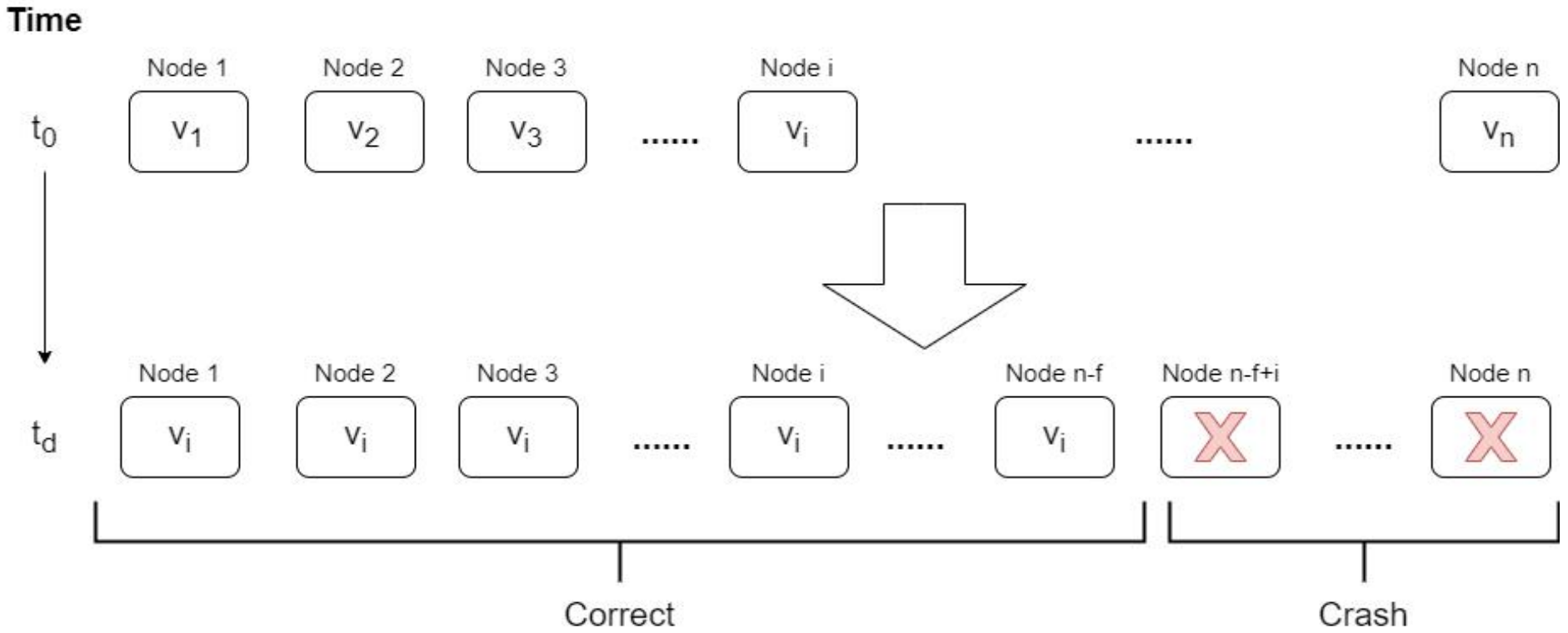
- Not sending any messages at all;
- Sending different and wrong messages to different neighbors;
- Lying about the input value;
- Colluding with other byzantine nodes.



Recall the Consensus

(Definition 3.1) Consensus

- There are n nodes which most f might crash.
- Each node i starts with the input value v_i, the nodes must decide a value, satisfying the following 3 properties:
- **Agreement, Termination, Validity**



Byzantine Agreement

(Definition 4.2) Byzantine Agreement

- **Finding consensus** as in **Definition 3.1** in a system with byzantine nodes is called byzantine agreement.
- An algorithm is **f-resilient** if it still works correctly with **f byzantine nodes.**

(Definition 3.1) Consensus

- There are n nodes which most f might crash.
- Each node i starts with the input value v_i, the nodes must decide a value, satisfying the following 3 properties:
- **Agreement, Termination, Validity**

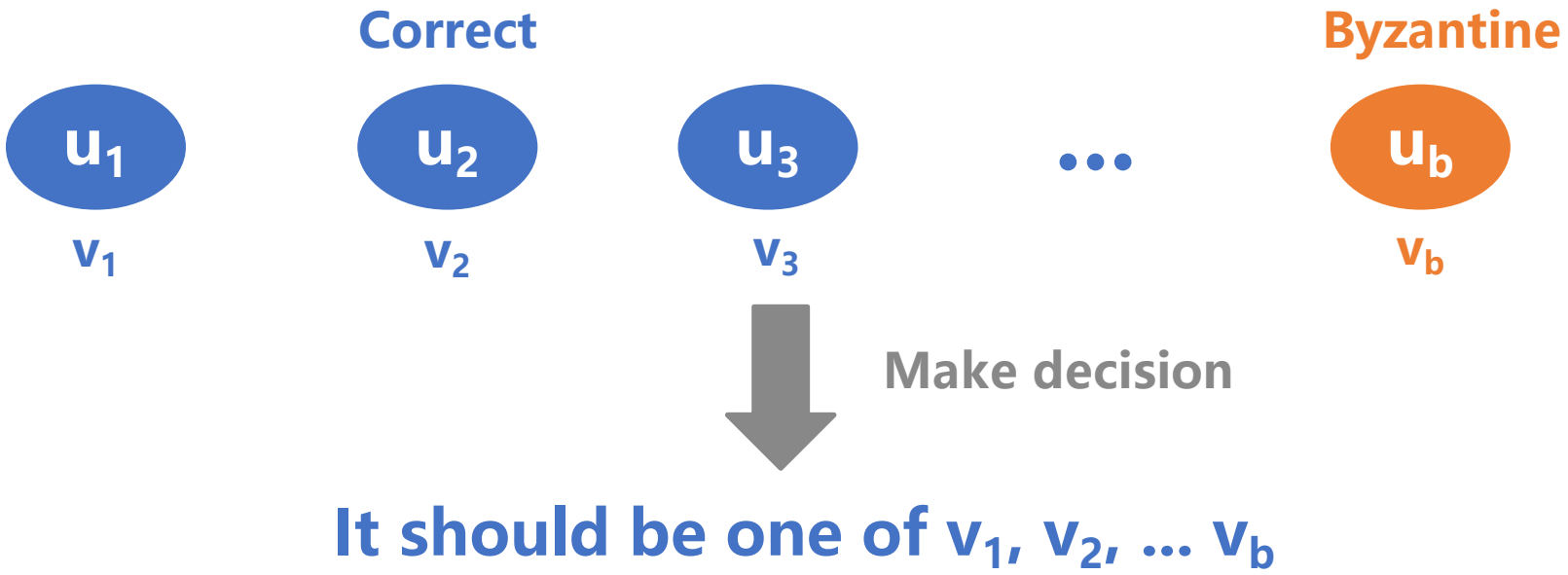
Agreement and termination are straight-forward, but what about validity?

Byzantine Agreement

Different Validities

(Definition 4.3) Any-Input Validity

- The decision value must be the **input value of any node.** (this is the definition we used in last chapter)
- That includes byzantine nodes, might not make sense (byzantine nodes may lie about their inputs)

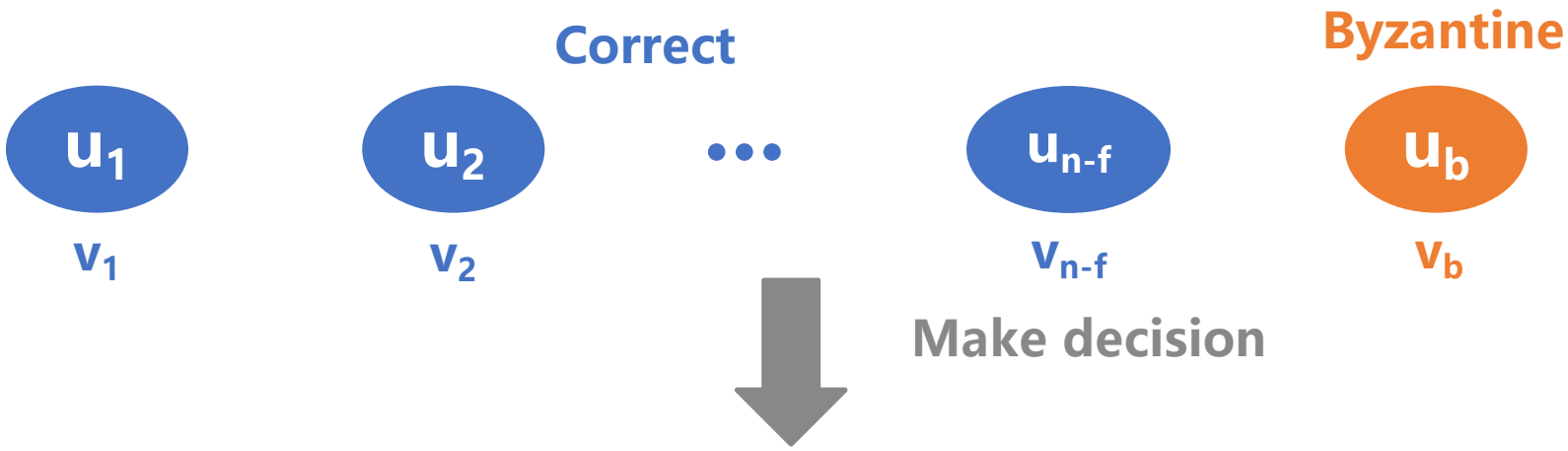


Byzantine Agreement

Different Validities

(Definition 4.4) Correct-Input Validity

- The decision value must be the **input value of a correct node.**
- Difficult because **byzantine node could behave like normal one just with different value**



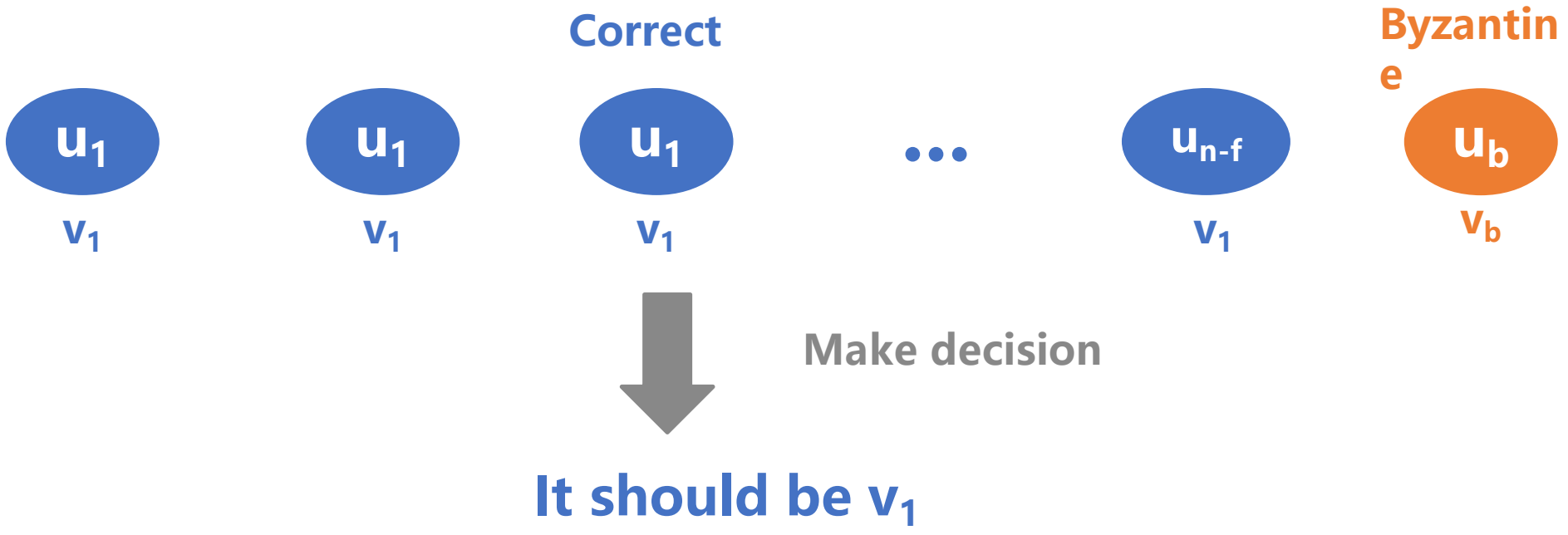
It should be one of v_1, v_2, … v_{n-f}

Byzantine Agreement

Different Validities

(Definition 4.5) All-Same Validity

- If all correct nodes **start with the same input v**, the decision value must be v.
- If the decision values are binary, then correct-input validity is induced by all same validity
- Not really useful **if the input values are not binary**



Byzantine Agreement

In the Synchronous Model

(Model 4.7) Synchronous

- In the synchronous model, **nodes operate in synchronous rounds.**
- In each round, each node may send a message to the other nodes, receive the messages sent by the other nodes, and do some local computation.

(Definition 4.8) Synchronous Runtime

- For algorithms in the synchronous model, the runtime is simply **the number of rounds from the start of the execution to its completion** in the worst case (every legal input, every execution scenario).

Byzantine Agreement

in the Synchronous Model

(Algorithm 4.9) Byzantine Agreement with f = 1.

1: Code for node u, with input value x:

Round 1

2: Send tuple(u, x) to all other nodes

3: Receive tuple(v, y) from all other nodes v

4: Store all received tuple(v, y) in a set S_u

Round 2

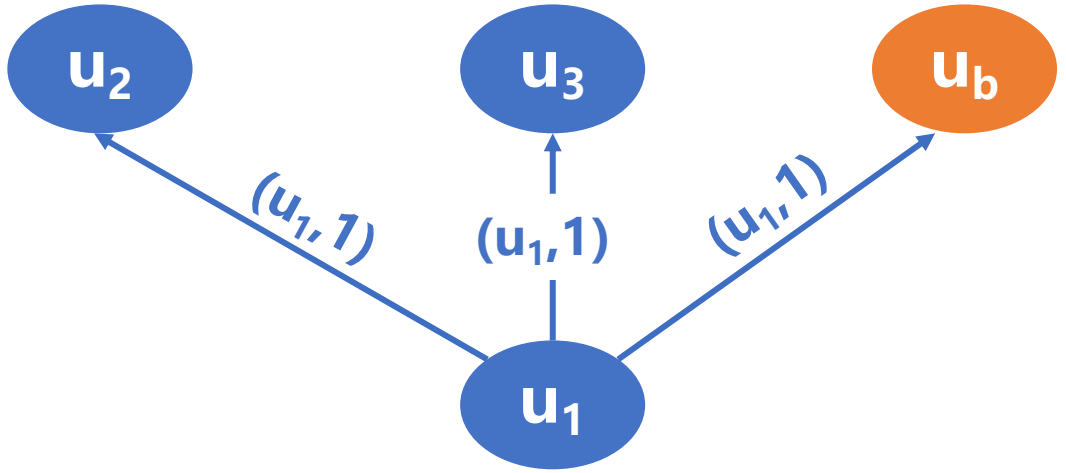
5: Send set S_u to all other nodes

6: Receive sets S_v from all nodes v

7: T = set of tuple(v, y) seen in at least two sets S_v, including own S_u

8: Let tuple(v, y) $\in$ T be the tuple with the smallest value y

9: Decide on value y



Byzantine Agreement

in the Synchronous Model

(Algorithm 4.9) Byzantine Agreement with f = 1.

1: Code for node u, with input value x:

Round 1

2: Send tuple(u, x) to all other nodes

3: Receive tuple(v, y) from all other nodes v

4: Store all received tuple(v, y) in a set S_u

Round 2

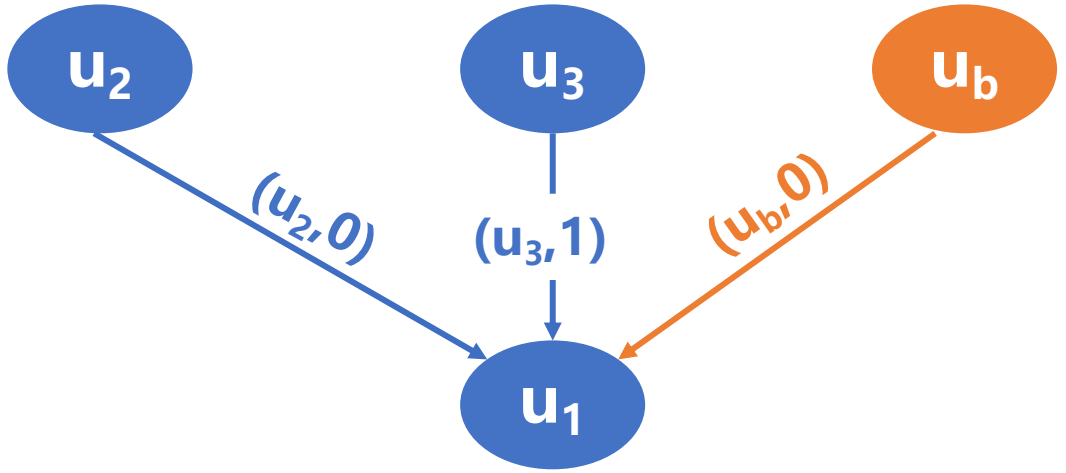
5: Send set S_u to all other nodes

6: Receive sets S_v from all nodes v

7: T = set of tuple(v, y) seen in at least two sets S_v, including own S_u

8: Let tuple(v, y) $\in$ T be the tuple with the smallest value y

9: Decide on value y



Byzantine Agreement

in the Synchronous Model

(Algorithm 4.9) Byzantine Agreement with f = 1.

1: Code for node u, with input value x:

Round 1

2: Send tuple(u, x) to all other nodes

3: Receive tuple(v, y) from all other nodes v

4: Store all received tuple(v, y) in a set S_u

Round 2

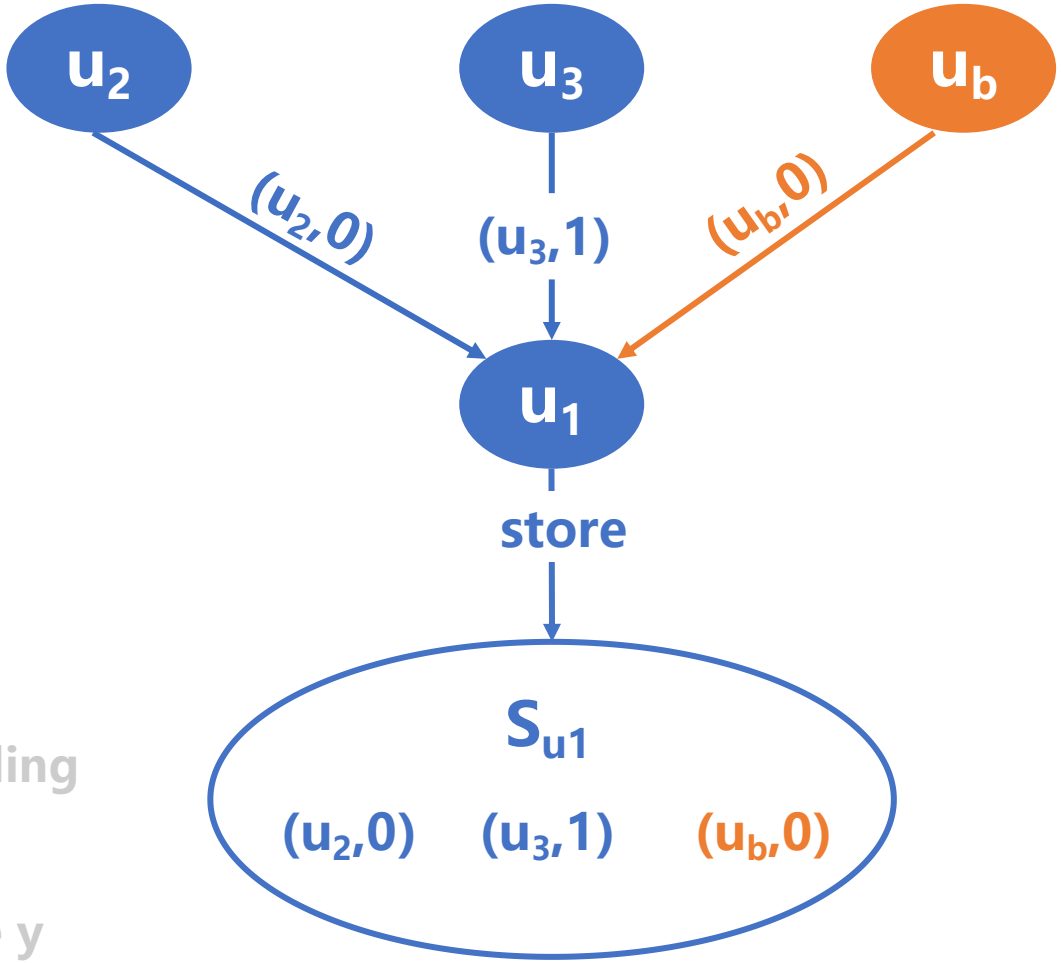
5: Send set S_u to all other nodes

6: Receive sets S_v from all nodes v

7: T = set of tuple(v, y) seen in at least two sets S_v, including own S_u

8: Let tuple(v, y) ∈ T be the tuple with the smallest value y

9: Decide on value y



Byzantine Agreement

in the Synchronous Model

(Algorithm 4.9) Byzantine Agreement with f = 1.

1: Code for node u, with input value x:

Round 1

2: Send tuple(u, x) to all other nodes

3: Receive tuple(v, y) from all other nodes v

4: Store all received tuple(v, y) in a set S_u

Round 2

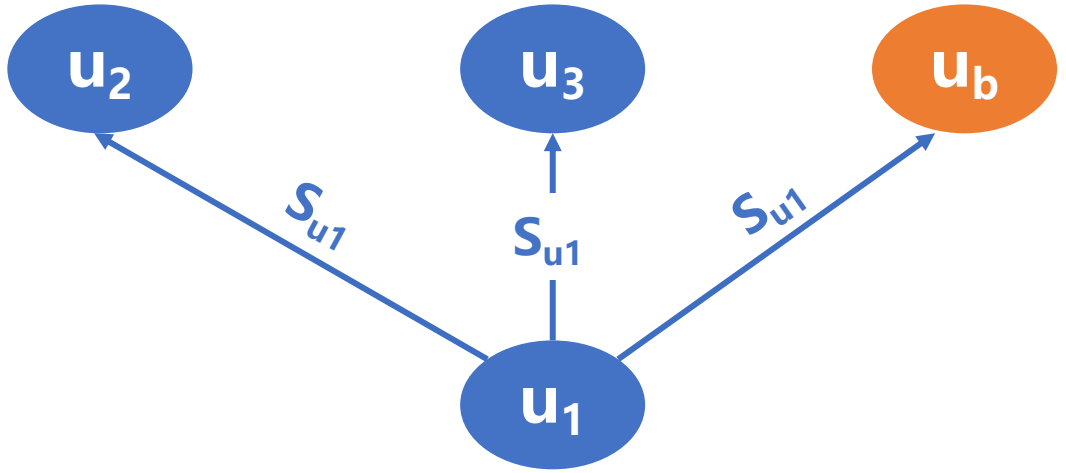
5: Send set S_u to all other nodes

6: Receive sets S_v from all nodes v

7: T = set of tuple(v, y) seen in at least two sets S_v, including own S_u

8: Let tuple(v, y) $\in$ T be the tuple with the smallest value y

9: Decide on value y



Byzantine Agreement

in the Synchronous Model

(Algorithm 4.9) Byzantine Agreement with f = 1.

1: Code for node u, with input value x:

Round 1

2: Send tuple(u, x) to all other nodes

3: Receive tuple(v, y) from all other nodes v

4: Store all received tuple(v, y) in a set S_u

Round 2

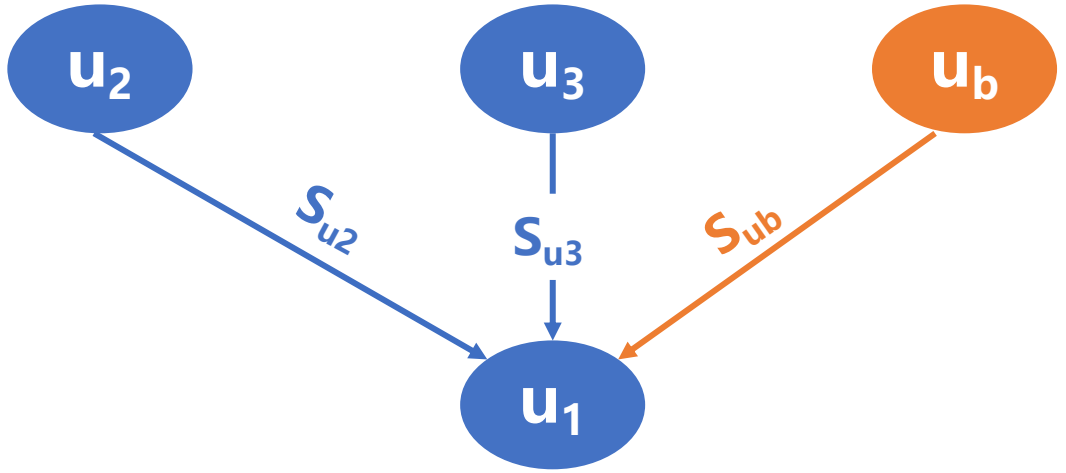
5: Send set S_u to all other nodes

6: Receive sets S_v from all nodes v

7: T = set of tuple(v, y) seen in at least two sets S_v, including own S_u

8: Let tuple(v, y) $\in$ T be the tuple with the smallest value y

9: Decide on value y



Byzantine Agreement

in the Synchronous Model

(Algorithm 4.9) Byzantine Agreement with f = 1.

1: Code for node u, with input value x:

Round 1

2: Send tuple(u, x) to all other nodes

3: Receive tuple(v, y) from all other nodes v

4: Store all received tuple(v, y) in a set S_u

Round 2

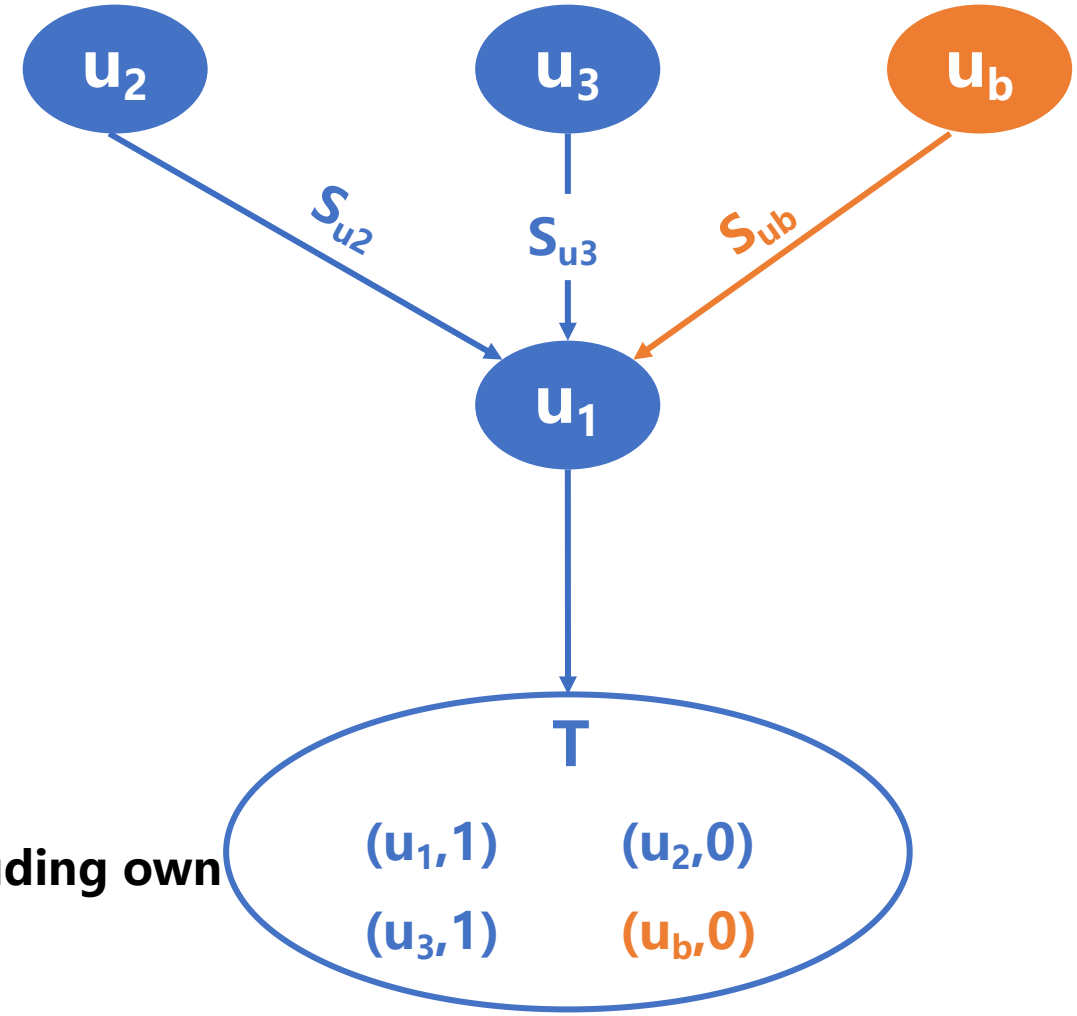
5: Send set S_u to all other nodes

6: Receive sets S_v from all nodes v

7: T = set of tuple(v, y) seen in at least two sets S_v, including own S_u

8: Let tuple(v, y) $\in$ T be the tuple with the smallest value **y**

9: Decide on value y



Byzantine Agreement

in the Synchronous Model

(Algorithm 4.9) Byzantine Agreement with f = 1.

1: Code for node u, with input value x:

Round 1

2: Send tuple(u, x) to all other nodes

3: Receive tuple(v, y) from all other nodes v

4: Store all received tuple(v, y) in a set S_u

Round 2

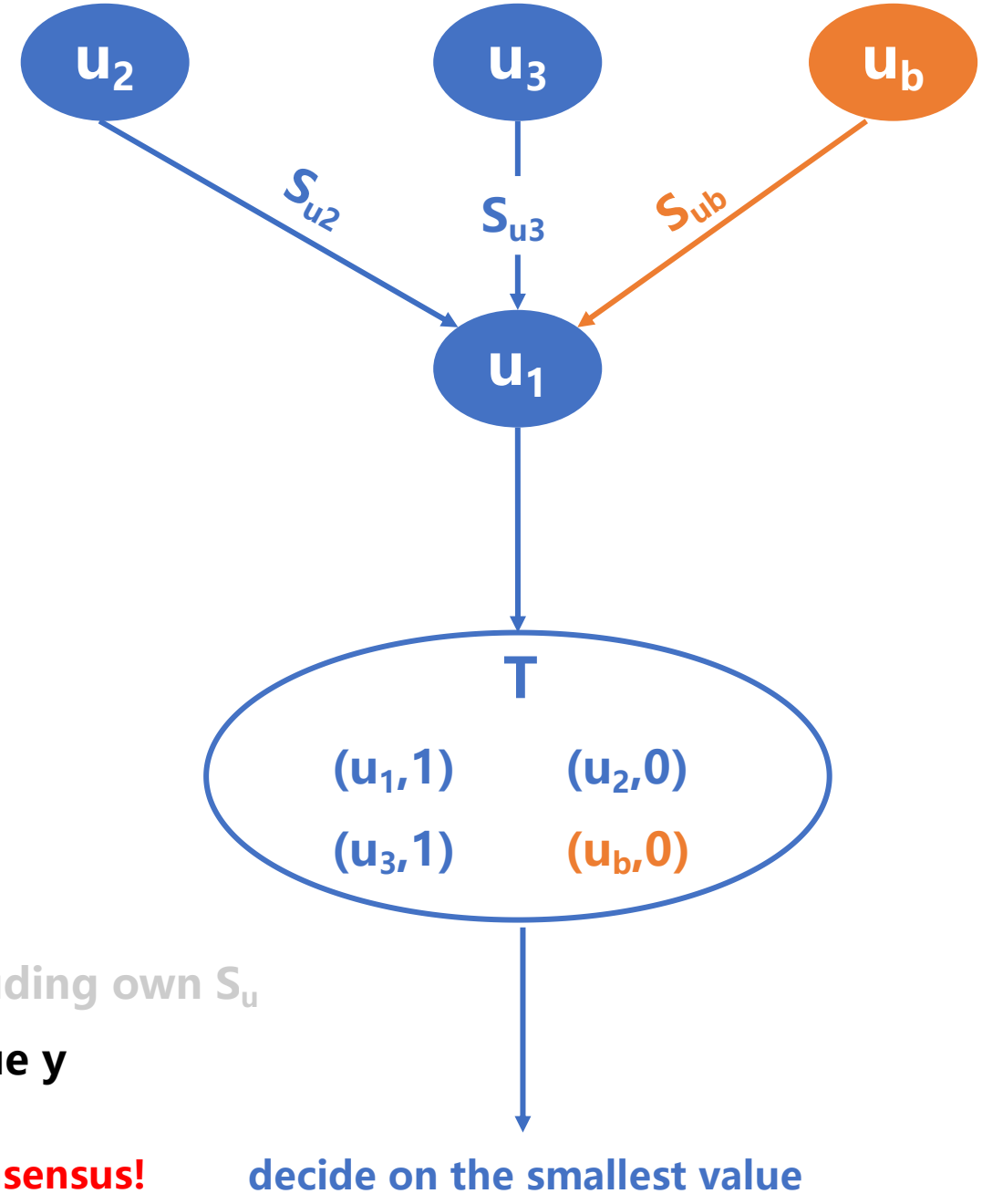
5: Send set S_u to all other nodes

6: Receive sets S_v from all nodes v

7: T = set of tuple(v, y) seen in at least two sets S_v, including own S_u

8: Let tuple(v, y) ∈ T be the tuple with the smallest value y

9: Decide on value y



Byzantine Agreement

in the Synchronous Model

Lemma 4.10 **If n ≥ 4, all correct nodes have the same set T.**

Proof

A correct node will see every correct value at least twice, once directly from another correct node, and once through the third correct node. So all correct values are in T.

If the byzantine node sends the same value to at least 2 other (correct) nodes, all correct nodes will see the value twice, so all add it to set T.

Algorithm 4.9 Byzantine Agreement with $f = 1$.

1: Code for node u, with input value x:

Round 1

2: Send $\texttt{tuple}(u, x)$ to all other nodes
3: Receive $\texttt{tuple}(v, y)$ from all other nodes v
4: Store all received $\texttt{tuple}(v, y)$ in a set S_u

Round 2

5: Send set S_u to all other nodes
6: Receive sets S_v from all nodes v
7: T = set of $\texttt{tuple}(v, y)$ seen in at least two sets S_v, including own S_u
8: Let $\texttt{tuple}(v, y) \in T$ be the tuple with the smallest value y
9: Decide on value y

Byzantine Agreement

in the Synchronous Model

Lemma 4.10 **If n ≥ 4, all correct nodes have the same set T.**

Proof

A correct node will see every correct value at least twice, once directly from another correct node, and once through the third correct node. So all correct values are in T.

If the byzantine node sends the same value to at least 2 other (correct) nodes, all correct nodes will see the value twice, so all add it to set T.

Algorithm 4.9 Byzantine Agreement with $f = 1$.

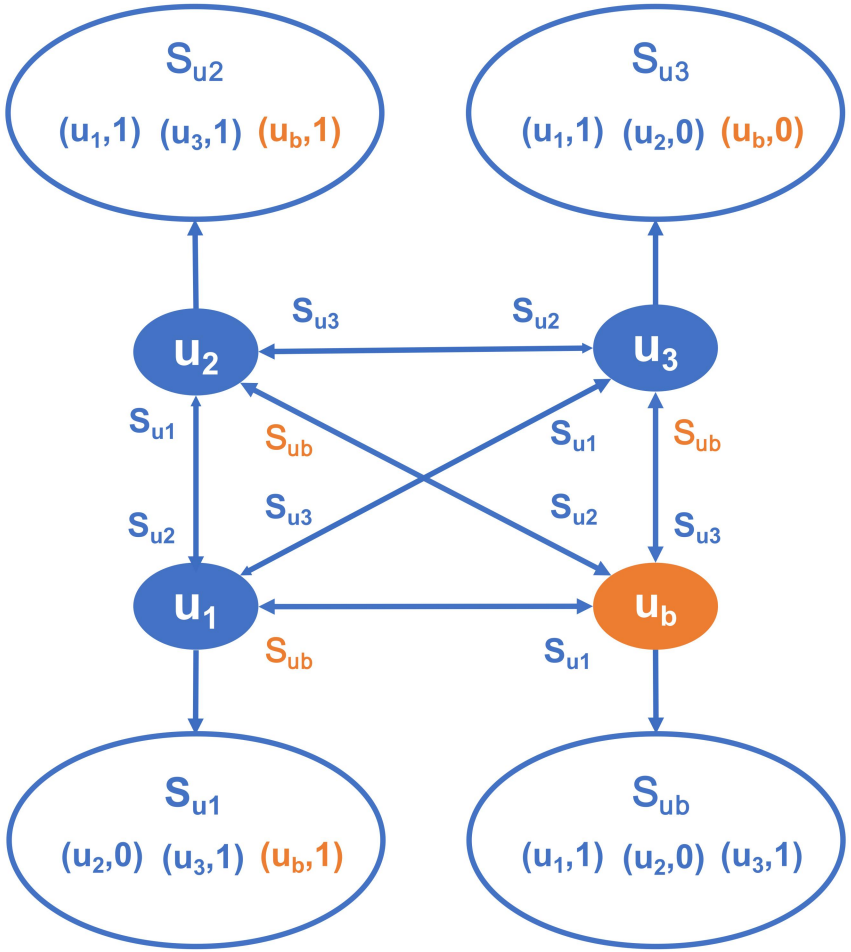
1: Code for node u, with input value x:

Round 1

2: Send $\texttt{tuple}(u, x)$ to all other nodes
3: Receive $\texttt{tuple}(v, y)$ from all other nodes v
4: Store all received $\texttt{tuple}(v, y)$ in a set S_u

Round 2

5: Send set S_u to all other nodes
6: Receive sets S_v from all nodes v
7: T = set of $\texttt{tuple}(v, y)$ seen in at least two sets S_v, including own S_u
8: Let $\texttt{tuple}(v, y) \in T$ be the tuple with the smallest value y
9: Decide on value y



Byzantine Agreement

in the Synchronous Model

Lemma 4.10 **If n ≥ 4, all correct nodes have the same set T.**

Proof

A correct node will see every correct value at least twice, once directly from another correct node, and once through the third correct node. So all correct values are in T.

If the byzantine node sends the same value to at least 2 other (correct) nodes, all correct nodes will see the value twice, so all add it to set T.

Algorithm 4.9 Byzantine Agreement with $f = 1$.

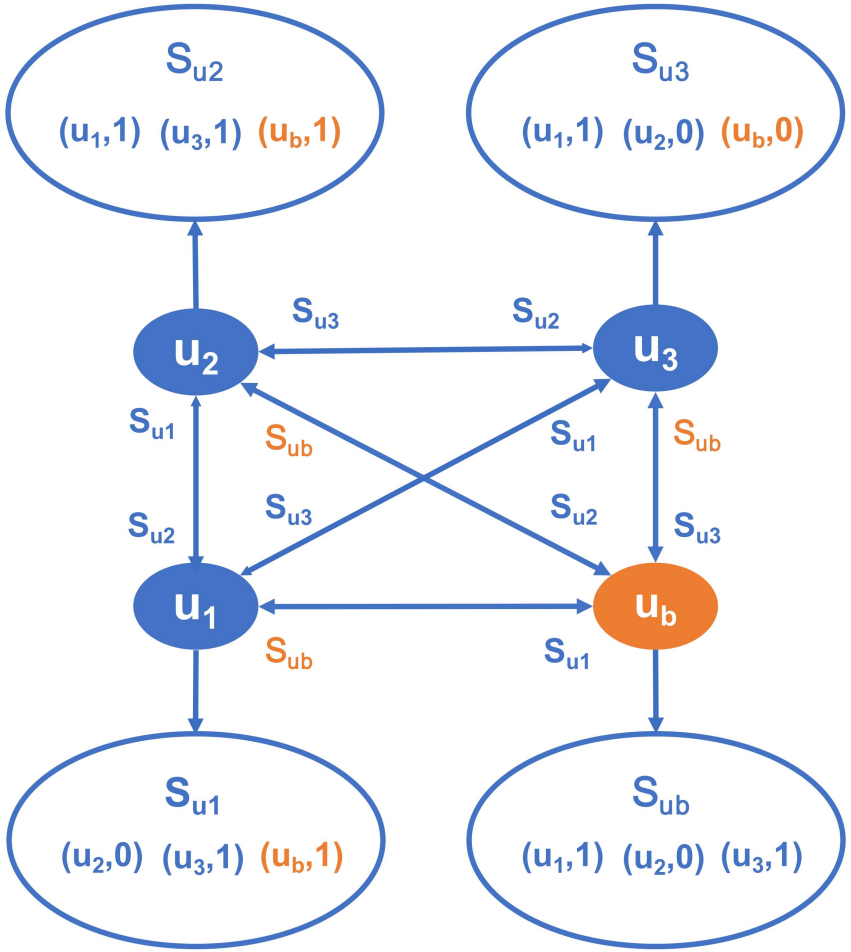
1: Code for node u, with input value x:

Round 1

2: Send $\texttt{tuple}(u, x)$ to all other nodes
3: Receive $\texttt{tuple}(v, y)$ from all other nodes v
4: Store all received $\texttt{tuple}(v, y)$ in a set S_u

Round 2

5: Send set S_u to all other nodes
6: Receive sets S_v from all nodes v
7: T = set of $\texttt{tuple}(v, y)$ seen in at least two sets S_v, including own S_u
8: Let $\texttt{tuple}(v, y) \in T$ be the tuple with the smallest value y
9: Decide on value y



Byzantine Agreement

in the Synchronous Model

Lemma 4.10 **If n ≥ 4, all correct nodes have the same set T.**

Proof

A correct node will see every correct value at least twice, once directly from another correct node, and once through the third correct node. So all correct values are in T.

If the byzantine node sends the same value to at least 2 other (correct) nodes, all correct nodes will see the value twice, so all add it to set T.

Algorithm 4.9 Byzantine Agreement with $f = 1$.

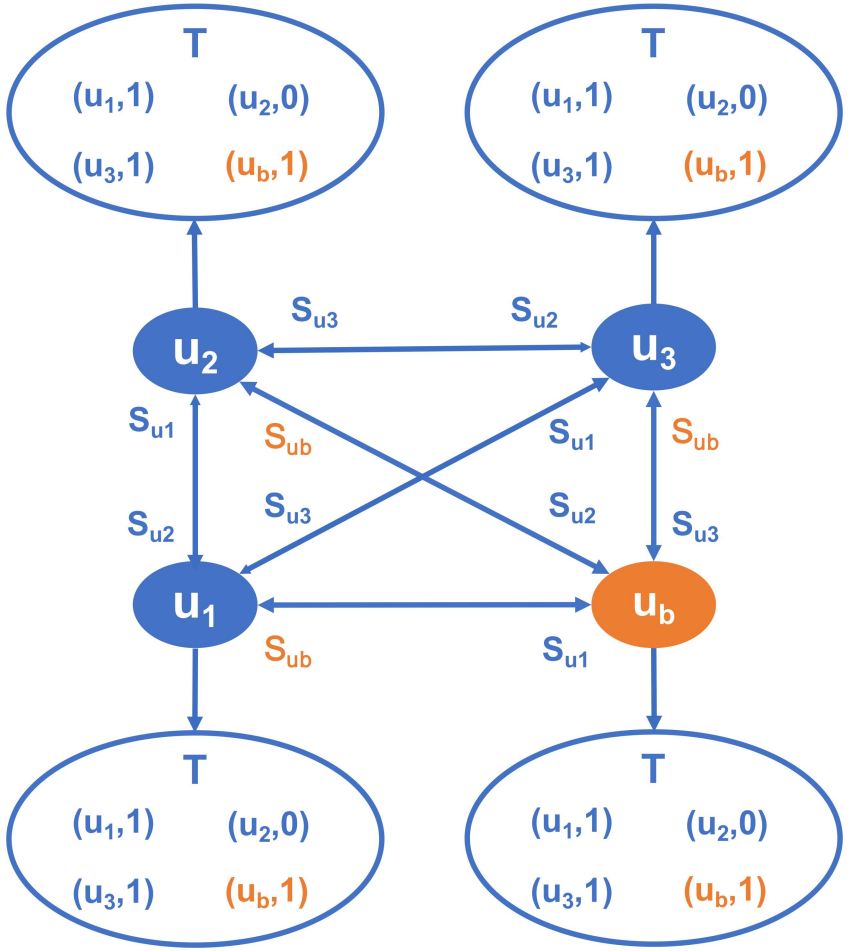
1: Code for node u, with input value x:

Round 1

2: Send $\texttt{tuple}(u, x)$ to all other nodes
3: Receive $\texttt{tuple}(v, y)$ from all other nodes v
4: Store all received $\texttt{tuple}(v, y)$ in a set S_u

Round 2

5: Send set S_u to all other nodes
6: Receive sets S_v from all nodes v
7: T = set of $\texttt{tuple}(v, y)$ seen in at least two sets S_v, including own S_u
8: Let $\texttt{tuple}(v, y) \in T$ be the tuple with the smallest value y
9: Decide on value y



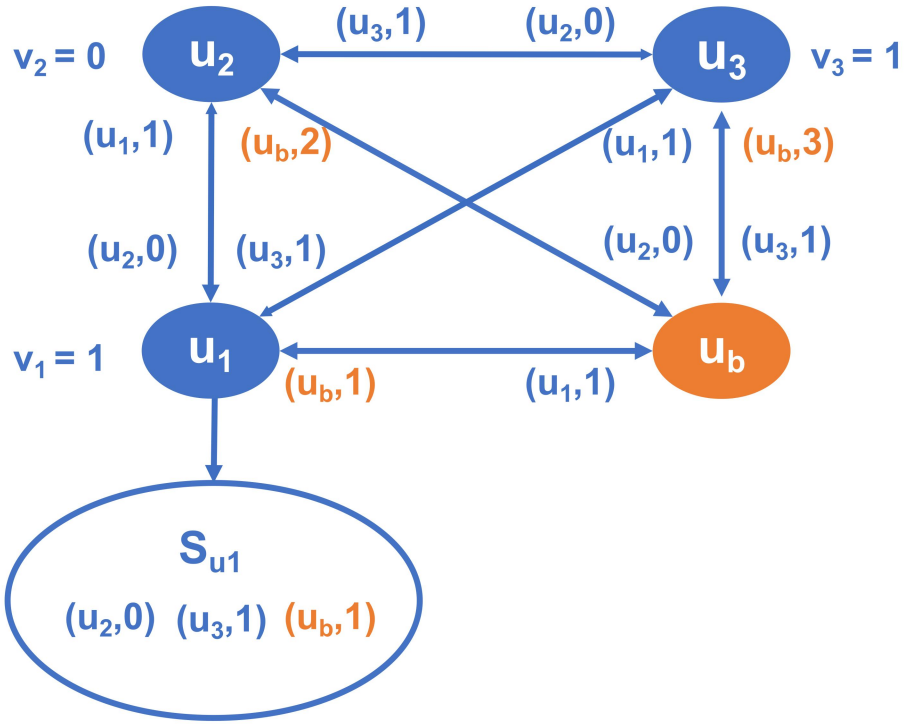
Byzantine Agreement

in the Synchronous Model

Lemma 4.10 **If n ≥ 4, all correct nodes have the same set T.**

Proof

If the byzantine node sends all different values to the correct nodes, none of these values will end up in any set T.



Algorithm 4.9 Byzantine Agreement with $f = 1$.

1: Code for node u, with input value x:

Round 1

2: Send $\texttt{tuple}(u, x)$ to all other nodes
3: Receive $\texttt{tuple}(v, y)$ from all other nodes v
4: Store all received $\texttt{tuple}(v, y)$ in a set S_u

Round 2

5: Send set S_u to all other nodes
6: Receive sets S_v from all nodes v
7: T = set of $\texttt{tuple}(v, y)$ seen in at least two sets S_v, including own S_u
8: Let $\texttt{tuple}(v, y) \in T$ be the tuple with the smallest value y
9: Decide on value y

Byzantine Agreement

in the Synchronous Model

Lemma 4.10 If n ≥ 4, all correct nodes have the same set T.

Proof

If the byzantine node sends all different values to the correct nodes, none of these values will end up in any set T.

Algorithm 4.9 Byzantine Agreement with $f = 1$.

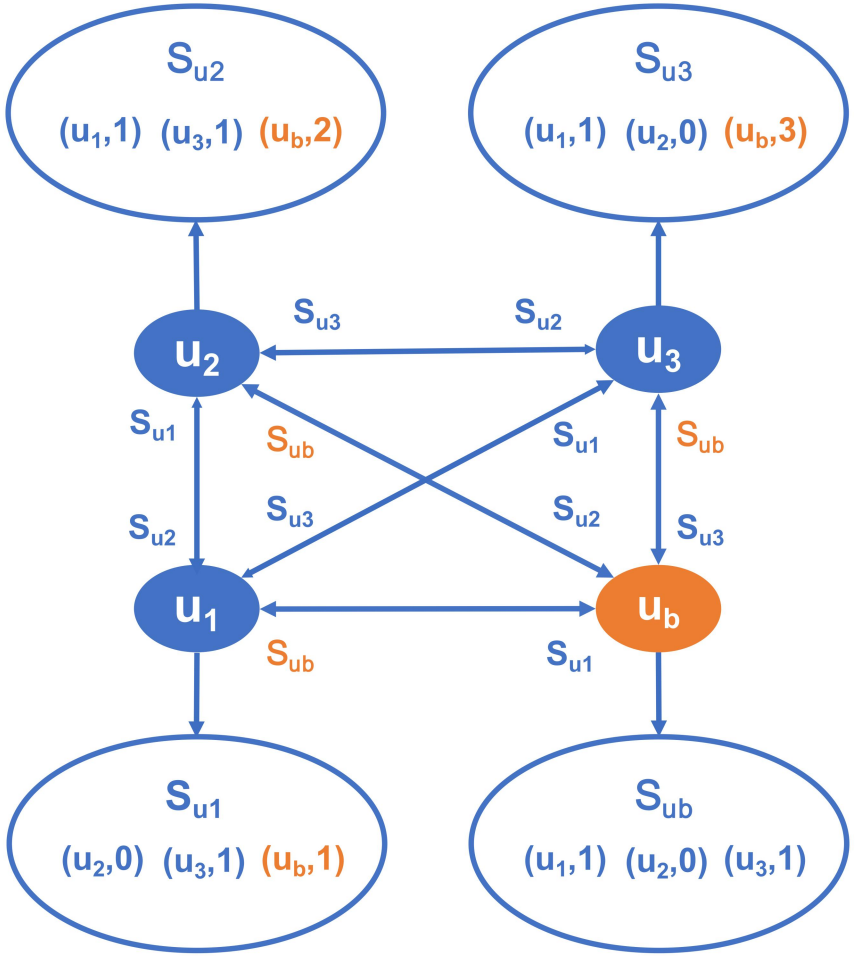
1: Code for node u, with input value x:

Round 1

2: Send $\texttt{tuple}(u, x)$ to all other nodes
3: Receive $\texttt{tuple}(v, y)$ from all other nodes v
4: Store all received $\texttt{tuple}(v, y)$ in a set S_u

Round 2

5: Send set S_u to all other nodes
6: Receive sets S_v from all nodes v
7: T = set of $\texttt{tuple}(v, y)$ seen in at least two sets S_v, including own S_u
8: Let $\texttt{tuple}(v, y) \in T$ be the tuple with the smallest value y
9: Decide on value y



Byzantine Agreement

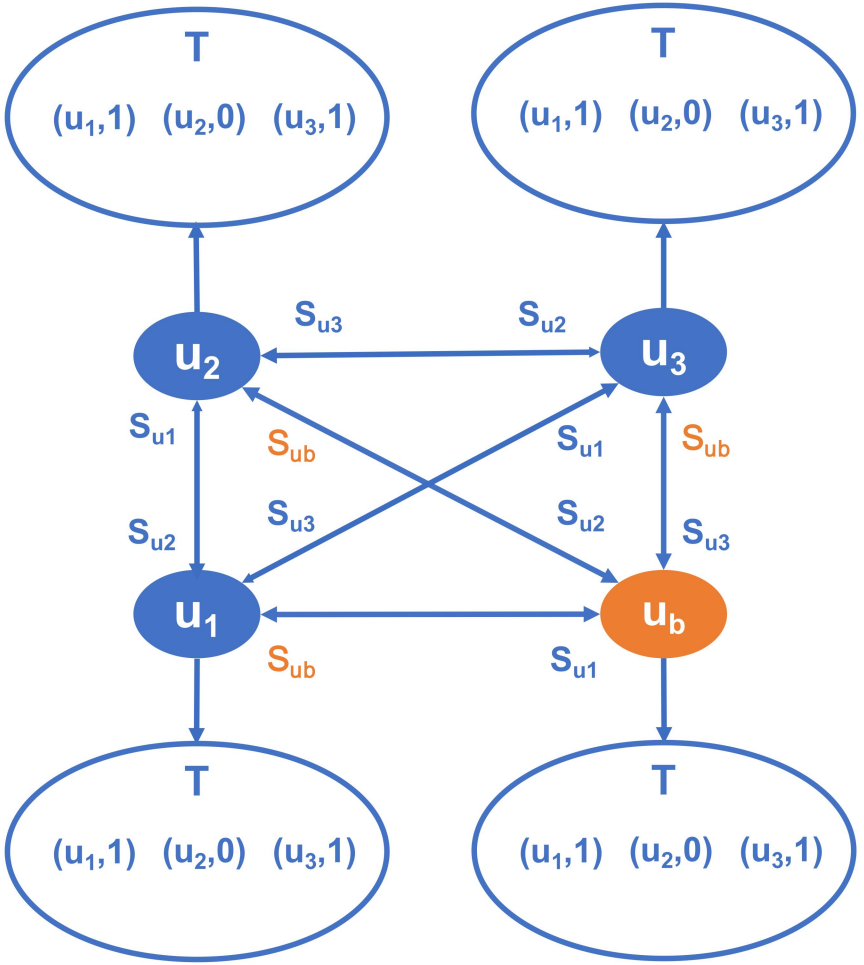
in the Synchronous Model

Lemma 4.10 **If n ≥ 4, all correct nodes have the same set T.**

Proof

A correct node will see every correct value at least twice, once directly from another correct node, and once through the third correct node. So all correct values are in T.

If the byzantine node sends all different values to the correct nodes, none of these values will end up in any set T.



Algorithm 4.9 Byzantine Agreement with $f = 1$.

1: Code for node u, with input value x:

Round 1

2: Send $\texttt{tuple}(u, x)$ to all other nodes
3: Receive $\texttt{tuple}(v, y)$ from all other nodes v
4: Store all received $\texttt{tuple}(v, y)$ in a set S_u

Round 2

5: Send set S_u to all other nodes
6: Receive sets S_v from all nodes v
7: T = set of $\texttt{tuple}(v, y)$ seen in at least two sets S_v, including own S_u
8: Let $\texttt{tuple}(v, y) \in T$ be the tuple with the smallest value y
9: Decide on value y

Byzantine Agreement

in the Synchronous Model

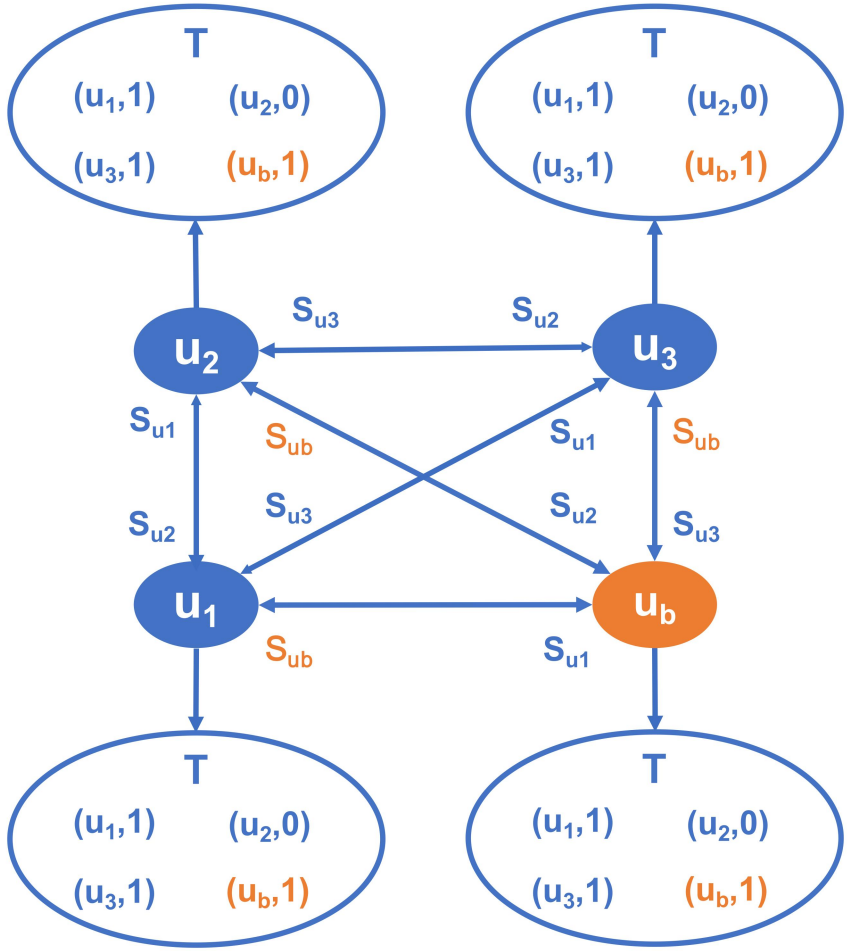
Theorem 4.11 **Algorithm 4.9 reaches byzantine agreement if n ≥ 4.**

Proof

- With Lemma 4.10 we know that all correct nodes have the same set T, and therefore agree on the same minimum value.
- The nodes agree on a value proposed by any node, so any-input validity holds.

Agreement

Validity



Byzantine Agreement

in the Synchronous Model

Theorem 4.11 **Algorithm 4.9 reaches byzantine agreement if n ≥ 4.**

Proof

- The algorithm terminates after two rounds.

Termination

Algorithm 4.9 Byzantine Agreement with $f = 1$.

1: Code for node u, with input value x:

Round 1

2: Send $\texttt{tuple}(u, x)$ to all other nodes
3: Receive $\texttt{tuple}(v, y)$ from all other nodes v
4: Store all received $\texttt{tuple}(v, y)$ in a set S_u

Round 2

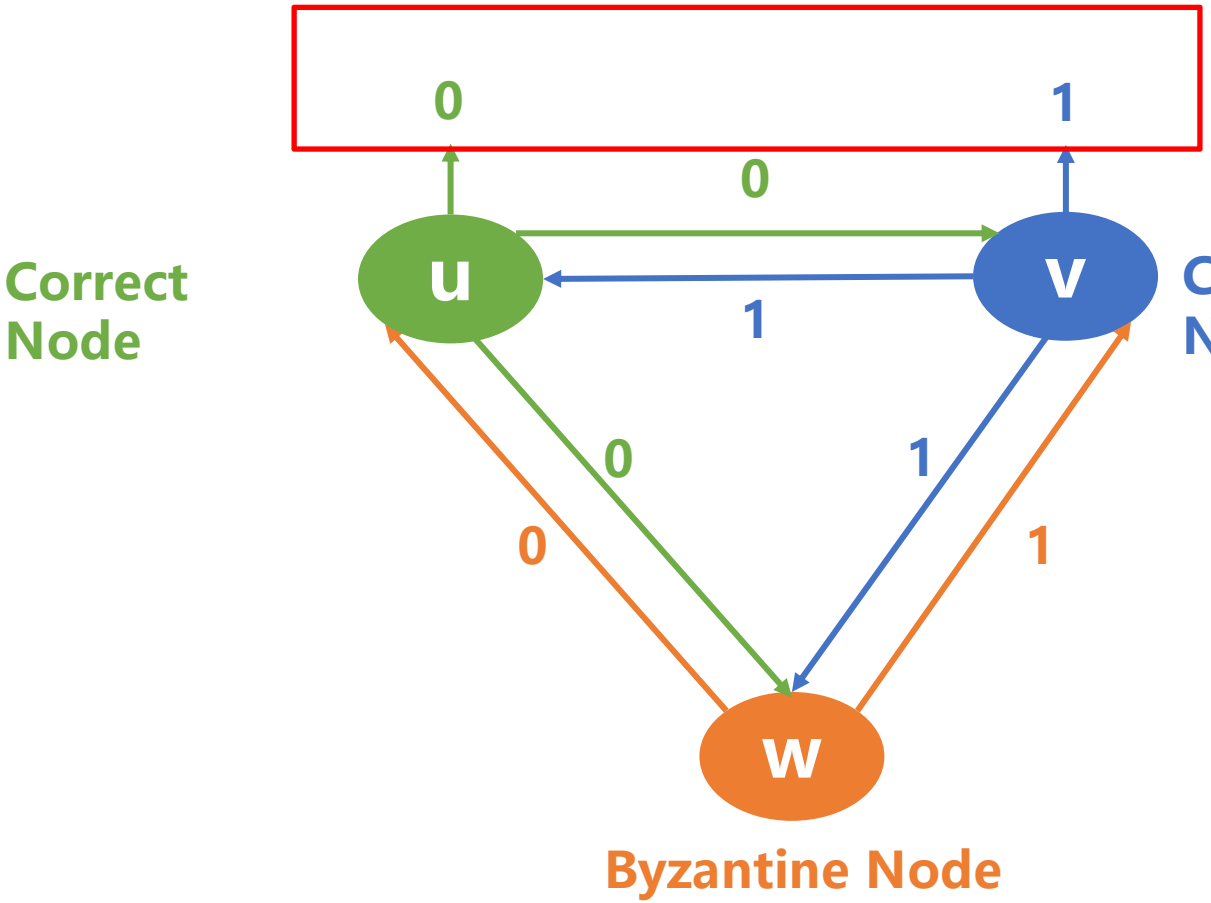
5: Send set S_u to all other nodes
6: Receive sets S_v from all nodes v
7: T = set of $\texttt{tuple}(v, y)$ seen in at least two sets S_v, including own S_u
8: Let $\texttt{tuple}(v, y) \in T$ be the tuple with the smallest value y
9: Decide on value y

Does Algorithm 4.9 also work with n = 3?

Byzantine Agreement in the Synchronous Model

Theorem 4.12 Three nodes cannot reach byzantine agreement with all-same validity if one node among them is byzantine.

Scenario 1:



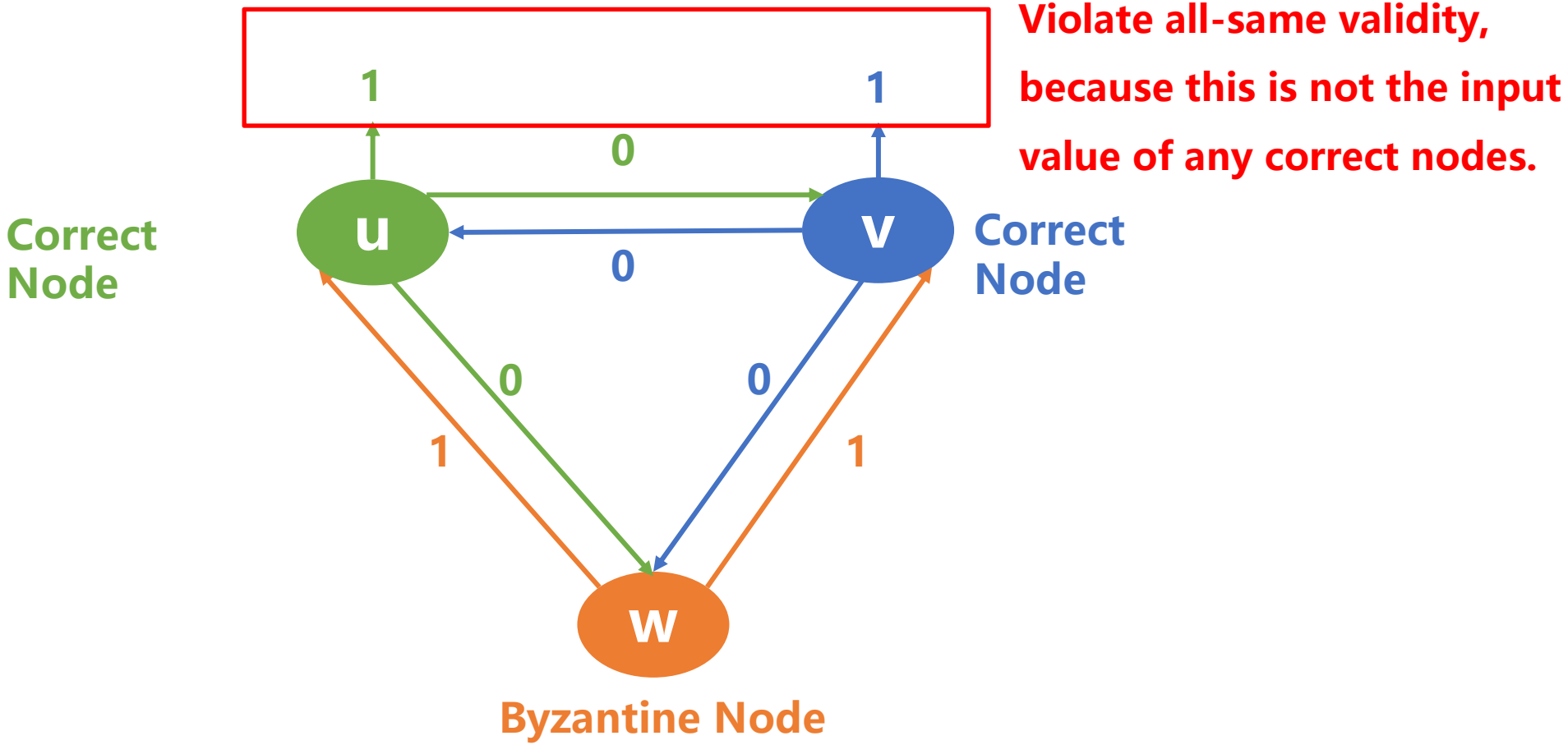
Fail to reach consensus!

And correct nodes cannot distinguish who is byzantine.

Byzantine Agreement in the Synchronous Model

Theorem 4.12 Three nodes cannot reach byzantine agreement with all-same validity if one node among them is byzantine.

Scenario 2:



Violate all-same validity, because this is not the input value of any correct nodes.

Byzantine Agreement in the Synchronous Model

Theorem 4.12

A network with n nodes cannot reach byzantine agreement with $f \geq n/3$ byzantine nodes.

King Algorithm in the Synchronous Model

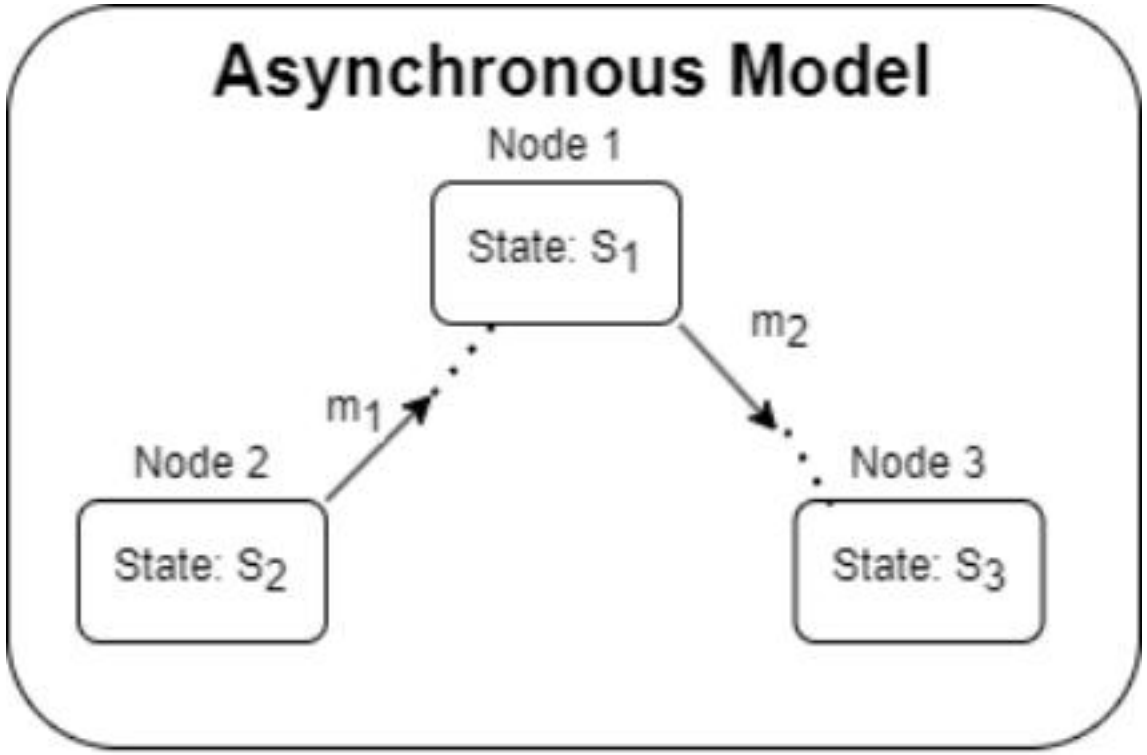
(Algorithm 4.14) King Algorithm (for f < n/3)

Algorithm 4.14 King Algorithm (for $f < n/3$)

```
1:  x = my input value
2:  for phase = 1 to f + 1 do
        Vote
3:      Broadcast value(x)
        Propose
4:      if some value(y) received at least n − f times then
5:          Broadcast propose(y)
6:      end if
7:      if some propose(z) received more than f times then
8:          x = z
9:      end if
        King
10:     Let node v_i be the predefined king of this phase i
11:     The king v_i broadcasts its current value w
12:     if received strictly less than n − f propose(y) then
13:         x = w
14:     end if
15: end for
```

Recall the Asynchronous Model

- Assumption: Messages **do not need to arrive at the same time** anymore. They have variable delays.
- Can we reach byzantine agreement in asynchronous model?



All nodes stop and wait until a message(m) arrives

Byzantine Agreement in the Asynchronous Model

(Algorithm 4.21) Asynchronous Byzantine Agreement (Ben-Or, for f < n/10)

Algorithm 4.21 Asynchronous Byzantine Agreement (Ben-Or, for $f < n/10$)

```
1:  x_u ∈ {0,1}                          ◁ input bit
2:  round = 1                            ◁ round
3:  while true do
4:     Broadcast propose(x_u,round)
5:     Wait until n − f propose messages of current round arrived
6:     if at least n/2 + 3f + 1 propose messages contain same value x then
7:        Broadcast propose(x,round + 1)
8:        Decide for x and terminate
9:     else if at least n/2 + f + 1 propose messages contain same value x then
10:       x_u = x
11:    else
12:       choose x_u randomly, with Pr[x_u = 0] = Pr[x_u = 1] = 1/2
13:    end if
14:    round = round + 1
15: end while
```

PBFT Solution

Appears in the *Proceedings of the Third Symposium on Operating Systems Design and Implementation, New Orleans, USA, February 1999*

Practical Byzantine Fault Tolerance

Miguel Castro and Barbara Liskov
Laboratory for Computer Science,
Massachusetts Institute of Technology,
545 Technology Square, Cambridge, MA 02139
{castro,liskov}@lcs.mit.edu

Abstract

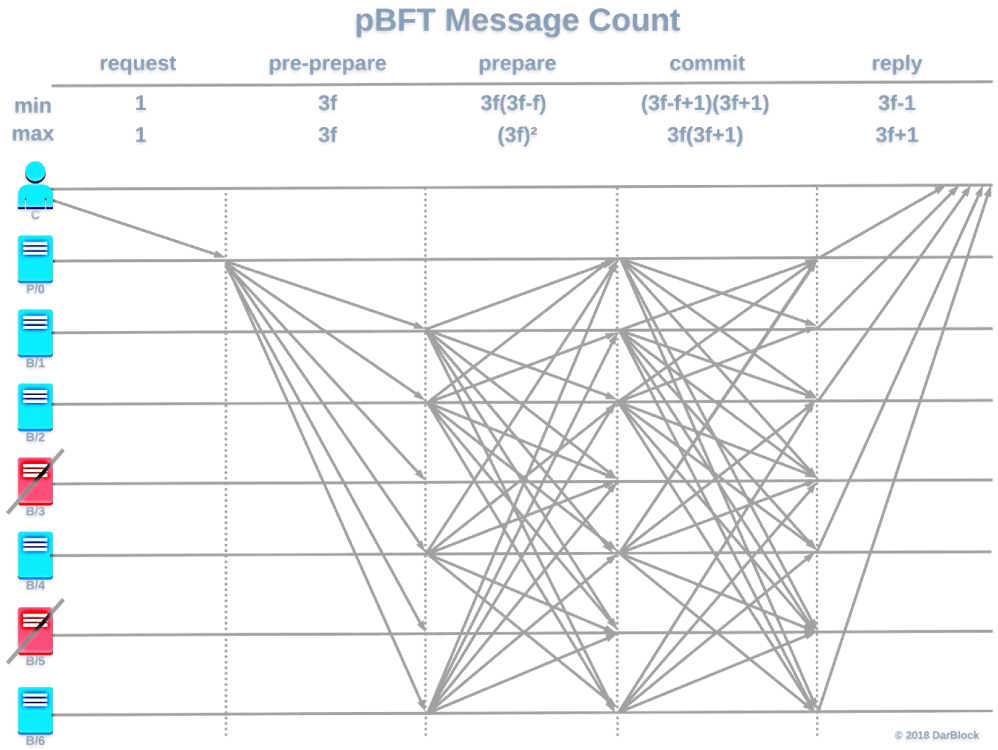
This paper describes a new replication algorithm that is able to tolerate Byzantine faults. We believe that Byzantine-fault-tolerant algorithms will be increasingly important in the future because malicious attacks and software errors are increasingly common and can cause faulty nodes to exhibit arbitrary behavior. Whereas previous algorithms assumed a synchronous system or were too slow to be used in practice, the algorithm described in this paper is practical: it works in asynchronous environments like the Internet and incorporates several important optimizations that improve the response time of previous algorithms by more than an order of magnitude. We implemented a Byzantine-fault-tolerant NFS service using our algorithm and measured its performance. The results show that our service is only 3% slower than a standard unreplicated NFS.

and replication techniques that tolerate Byzantine faults (starting with [19]). However, most earlier work (e.g., [3, 24, 10]) either concerns techniques designed to demonstrate theoretical feasibility that are too inefficient to be used in practice, or assumes synchrony, i.e., relies on known bounds on message delays and process speeds. The systems closest to ours, Rampart [30] and SecureRing [16], were designed to be practical, but they rely on the synchrony assumption for correctness, which is dangerous in the presence of malicious attacks. An attacker may compromise the safety of a service by delaying non-faulty nodes or the communication between them until they are tagged as faulty and excluded from the replica group. Such a denial-of-service attack is generally easier than gaining control over a non-faulty node.

PBFT Solution

4 Phases Consensus:

1. The client sends a request to the primary node.
2. The primary node broadcasts the request to the all secondary nodes.
3. All nodes perform the service requested and then reply to the client.
4. The request is served successfully when the client receives ‘m+1’ same replies.



Drawback:

the **O(n²)** increasing message count as nodes are added to the set

PBFT is susceptible to *Sybil attacks*